Grounded: Klimaat, Bodem en Data: werken met R

Ronald M. Visser Marleen van Zon

2026-10-27

Table of contents

Voorwoord 4

1 Introductie 5

2 Werkcollege 1: de eerste stappen met R en Rstudio 6
- 2.1 Inleiding . . . 6
- 2.2 Project aanmaken . . . 6
- 2.3 Een eerste bestand aanmaken . . . 7
- 2.4 Start van een script met analyses . . . 7
- 2.5 Inlezen grondwater data . . . 9
- 2.6 Verkennen van de data . . . 12
- 2.7 Zelf aan de slag met een aantal opdrachten . . . 14
- 2.8 De eerste grafieken . . . 14
- 2.9 Zelf aan de slag met een aantal opdrachten . . . 21
- 2.10 Op zoek naar aanvullende data . . . 21
- 2.11 Zelf aan de slag met een aantal opdrachten . . . 23
- 2.12 Combineren van datasets . . . 24
- 2.13 Zelf aan de slag met een aantal opdrachten . . . 25
- 2.14 Huiswerk: verwerken in de eigen projecten . . . 25

3 Werkcollege 2: werken met weer- en klimaatdata 26
- 3.1 Inleiding . . . 26
- 3.2 Weerdata . . . 26
  - 3.2.1 Project en script aanmaken . . . 26
  - 3.2.2 Data inlezen en de eerste verkenning van de data . . . 27
- 3.3 Grafieken maken voor de neerslag per dag . . . 30
  - 3.3.1 Weer zelf aan de slag . . . 34
- 3.4 Temperatuur . . . 34
  - 3.4.1 Nu even zelf nadenken . . . 38
  - 3.4.2 Zelf aan de slag . . . 40
- 3.5 Grafieken combineren . . . 40
  - 3.5.1 Zelf aan de slag . . . 41
- 3.6 Klimaat: meerjarige data . . . 43
  - 3.6.1 Zelf doen: . . . 43
  - 3.6.2 En dan weer zelf aan de slag . . . 45

3.7 Historische klimaatdata toevoegen . . . 45
3.7.1 En nu zelf weer aan de slag. . . . 48
3.8 Project . . . 49

4 Werkcollege 3: werken met ruimtelijke data **50**
4.1 Inleiding . . . 50
4.2 Project aanmaken . . . 50
4.3 Bibliotheken . . . 50
4.4 Een eerste kaart maken . . . 51
4.4.1 Nu zelf nadenken . . . 60
4.5 Een kaart van Delft . . . 60
4.6 Grondwaterpunten in Delft op de kaart . . . 62
4.6.1 Nu zelf aan de slag . . . 68
4.7 Individuele meetreeksen downloaden . . . 68
4.7.1 Nu even zelf . . . 71
4.8 Kades in Delft selecteren . . . 71
4.8.1 Nu zelf aan de slag . . . 74
4.9 Tot slot . . . 74

References **75**

Voorwoord

Dit is een boek gemaakt middels Quarto (https://quarto.org/docs/books/). Het is gemaakt in het kader van de Saxion minor Grounded: klimaat, bodem en data (https://www.saxion.nl/opleidingen/minoren/g/grounded-klimaat-bodem-en-data)van Saxion. Deze minor wordt vormgegeven samen met de onderzoeksgroep GROUNDED (Ground Expertise by data: https://groundexpertisebydata.nl), waardoor we de koppeling onderzoek en onderwijs versterken. Daarnaast is dit ook een product van het door Comenius gefinancieerde project Kades als Duurzaam Erfgoed (KADE; https://doi.org/10.61686/CAOBG60422).

1 Introductie

Dit quarto-boek bevat een aantal oefeningen om te leren werken met R. Het bestaat uit drie verschillende werkcolleges:

- College 1
- College 2
- College 3

Voordat je aan college 1 begint, moet je eerst zorgen dat je R geïnstalleerd hebt. Zorg dat je R eerst download vanaf https://cran.r-project.org/. Installeer dit daarna.

We maken in dit werkcollege gebruik van RStudio. Nadat je R geïnstalleerd hebt, download je RStudio vanaf https://posit.co/downloads/ en installeer dit ook.

2 Werkcollege 1: de eerste stappen met R en Rstudio

2.1 Inleiding

We gaan eerst even leren werken in RStudio. Dit is een gratis en open-source geïntegreerde ontwikkelomgeving (IDE) voor R. Hierin is het belangrijk om met projecten te werken, omdat dit programmeren in R eenvoudiger en sneller maakt. Als je in een project werkt, wordt de werkdirectory automatisch ingesteld; iets wat je handmatig moet doen wanneer je R-scripts buiten projecten om schrijft. De eerste stap om aan een nieuw onderzoek te beginnen, is dus RStudio openen en vervolgens een nieuw project aan te maken.

2.2 Project aanmaken

Maak een nieuw project aan in een nieuwe map. Deze noemen we nu Grounded_WC1.

De stappen die je daarvoor moet zetten, staan heel goed uitgelegd op de website van de makers: https://support.posit.co/hc/en-us/articles/200526207-Using-RStudio-Projects. Volg die stappen dus voor deze opdracht.

> Tip
>
> Er zijn diverse instellingen in RStudio. Deze zijn te vinden via `Menu > Tools > Global Options...` Zorg dat je in ieder geval het automatisch opslaan van de workspace uitzet, omdat je dan iedere analyse met een verse omgeving begint. Dit voorkomt fouten.

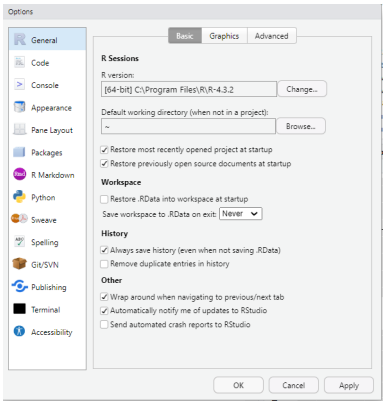


Figure 2.1: Instellingen van RStudio

2.3 Een eerste bestand aanmaken

Het project bevat nog geen bestanden. Daarom maken we nu een eerste script aan. Dat kan via het menu middels `File > New File > R Script` (of druk op CTRL + SHIFT + N).

In het lege script moeten jullie nu de eerste regel code schrijven

```
print("Hello World!")
```

Dit is de regel waarmee vrijwel iedere programmeeroefening begint. Je kunt dit bestand nu opslaan door op de save-knop te drukken of via CTRL + S het op te slaan. Kies een logische bestandsnaam, bijvoorbeeld: “Grounded_WC1.R”

Je kunt dit script nu als geheel uitvoeren door CTRL + SHIFT + S te typen of op de “Source”-knop te klikken. Als het goed is, krijg je nu de output “Hello World!” in het Console-scherm. Dat is jouw eerste script!

2.4 Start van een script met analyses

De regel met `hello world` was maar een voorbeeld en die hebben we niet meer nodig. Verwijder dus de tekst. De gaan nu een script maken om een simpele analyse uit te voeren. Daartoe hebben we een aantal libraries nodig. Libraries, ook wel packages genoemd, zijn collecties met kant en klare functies en datasets.

> **Tip**
>
> Ieder analyse-bestand begint altijd met het aanroepen van de benodigde bibliotheken en eventuele mappen die moeten worden aangemaakt. Hierdoor is de basis altijd goed terug te vinden. Soms kom je tijdens het schrijven van de analyses erachter dat een bepaalde library moet worden aangeroepen. Zorg dan dat die altijd bovenaan wordt ingevoegd.

Kopiëer de onderstaande code naar jouw script uit de voorgaande opdracht.

```
# load libraries
library(dplyr)
library(ggplot2)
library(lubridate)

# force non scientific formatting of numbers
options(scipen = 999)
options(digits=3)
```

> **Tip**
>
> Het hashtag symbool (#) gebruik je voor commentaren in de code. Dit wordt nooit uitgevoerd, maar vergroot de leesbaarheid van een script.

De kans is groot dat de eerste keer de libraries dplyr (Wickham, François, et al. 2023), ggplot (Wickham 2016) en lubridate (Grolemund and Wickham 2011) nog niet geïnstalleerd zijn. De installatie van packages staat uitgelegd op bijvoorbeeld deze locatie: https://libguides.chapman.edu/R/packages. Volg deze stappen indien nodig.

Naast het laden van libraries, stellen we ook de formatting van getallen in, door het aantal decimalen op 3 te zetten en getallen non-scientific te zetten.

We gaan nu eerst twee mappen aanmaken. De eerste `data` is voor het opslaan van de ruwe data. Daarnaast een map waarin we grafieken en dergelijke exporteren, die noemen we `export`.

```
# create folders
dir.create("data")
dir.create("export")
```

> **Tip**
>
> Hierna zullen we niet meer zeggen dat je code moet kopiëren. We gaan ervan uit dat je dat doet. Code is herkenbaar aan het eigen `lettertype`.

2.5 Inlezen grondwater data

Ga nu naar de website https://www.grondwatertools.nl/gwsinbeeld/. Zoek op de kaart naar Deventer en zoek het grondwaterpunt “B27G0060”.

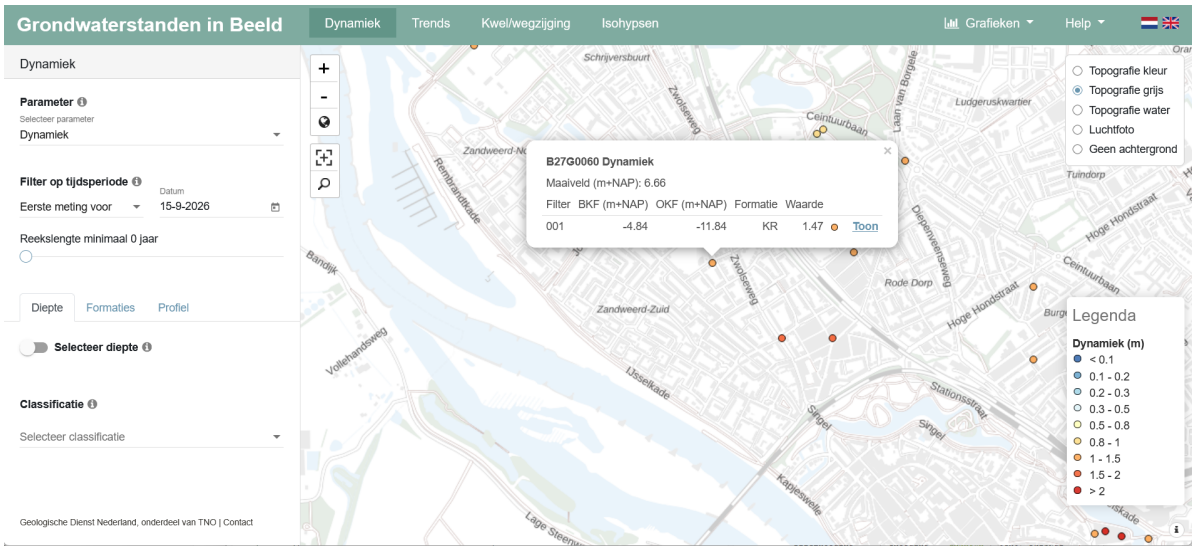


Figure 2.2: De website <https://www.grondwatertools.nl/gwsinbeeld/>{target=“_blank”} met daarin het grondwaterpunt B27G0318.

Als je nu op de link met “Toon” erop klikt, krijg je het onderstaande beeld. Bij het hamburgermenu kun je de optie “Download CSV” kiezen.

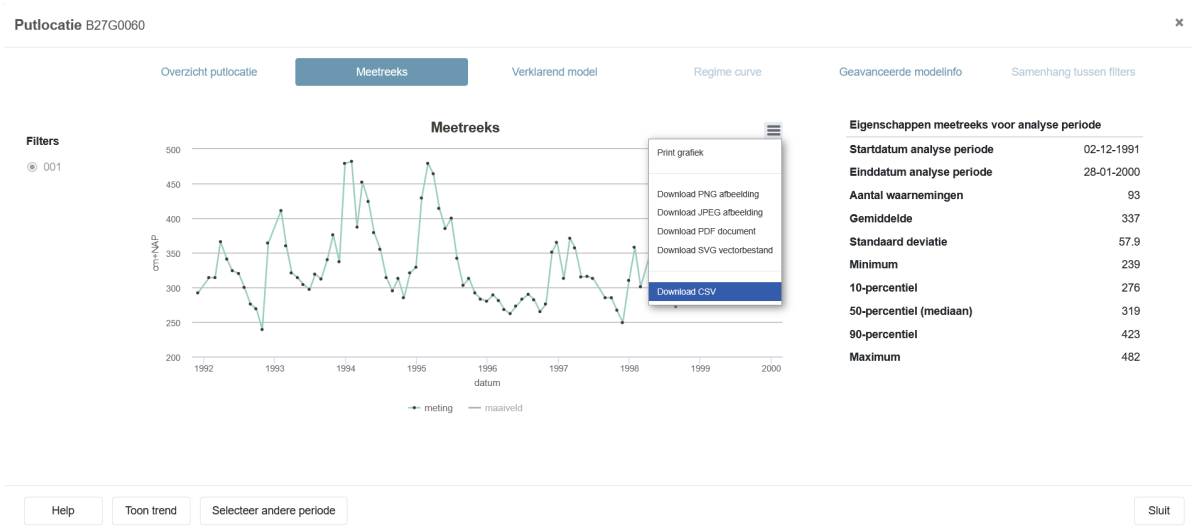


Figure 2.3: De grondwaterstanden van meetpunt B27G0060 op website <https://www.grondwatertools.nl/gwsinbeeld/>{target="_blank"}.

Download dit bestand en sla het op in de `data` map van het project.

> **Tip**
>
> We hebben eerder aangegeven dat je met `source` het hele script kunt uitvoeren. Dat is soms wat omslachtig en wil je alleen een enkele regel uitvoeren. Dat kan door een regel te selecteren en dan op `ctrl+enter` of via menu `Code > Run Selected Line(s)` te kiezen. Dan voer je alleen een enkele regel (of meerdere geselecteerde regels) uit.

De volgende stap is het inladen van dit databestand in de R-omgeving. Dat doen we met de functie `read.csv()` zoals in de onderstaande code is aangegeven. Je ziet ook een pijl staan: `<-`. Dat is de toewijzing aan een variabele. Deze regel betekent letterlijk: maak de variable `B27G0060001`aan door middel van het uitvoeren van de functie `read.csv("data/B27G0060001_meetreeks.csv")`. Dit is standaard-syntax in R.

```
# read data
B27G0060001 <- read.csv("data/B27G0060001_meetreeks.csv")
```

> **Tip**
>
> Een variabele is een container waarin data wordt opgeslagen (https://www.w3schools.com/r/r_variables.asp). Deze data een enkel getal of tekst zijn, maar ook een hele tabel, matrix of een ander soort object. Het is heel flexibel en de toekenning bepaalt het type.

De naamgeving van een variabele is aan regels gebonden, zie: https://www.w3schools.com/r/r_variables_name.asp. Let op! Een variabele kun je makkelijk overschrijven door deze opnieuw te definiëren.

Wanneer we nu willen weten welke data is ingelezen, kun je de data snel bekijken met de functie `head()`

```
head(B27G0060001)
```

```
    datum.meting
1 1991-12-02;292
2 1992-01-29;314
3 1992-02-28;314
4 1992-03-27;366
5 1992-04-28;341
6 1992-05-27;324
```

Dat ziet er niet goed uit. Als je het csv-bestand opent in een tekst-editor, zoals Notepad/Kladblok of Notepad++ (https://notepad-plus-plus.org/) zul je zien dat de data bestaat uit tekst en dat de ; de verschillende velden scheidt (bijvoorbeeld: `1991-12-02;292`).

De functie `read.csv()` gaat ervanuit dat een csv uit "comma-separated values" bestaat (https://nl.wikipedia.org/wiki/Kommagescheiden_bestand), vandaar ook de bestandsnaam. De leveranciers van de waterstanden gebruiken echter een puntkomma (;). Gelukkig is de functie flexibel en kun je dit als optie invoeren bij de functie. Als je de code aanpast naar de onderstaande tekst, zal de data wel goed ingelezen worden.

```
# read data
B27G0060001 <- read.csv("data/B27G0060001_meetreeks.csv", sep = ";")
head(B27G0060001)
```

```
       datum meting
1 1991-12-02    292
2 1992-01-29    314
3 1992-02-28    314
4 1992-03-27    366
5 1992-04-28    341
6 1992-05-27    324
```

2.6 Verkennen van de data

Nu de data ingelezen is, kunnen we een aantal zaken te weten komen, zoals de gemiddelde grondwaterstand en bijbehorende standaarddeviatie, en bijvoorbeeld de maximale en minimale standen. Daartoe willen we eigenlijk alleen in de kolom `meting`van ons data.frame kijken. Dat doe je door de variable te noemen met daaraan vast een $-teken en dan de naam van de kolom: `B27G0060001$meting`

```
# minimale grondwaterstand
min(B27G0060001$meting)
```

```
[1] 239
```

```
# maximale grondwaterstand
max(B27G0060001$meting)
```

```
[1] 482
```

```
# gemiddelde grondwaterstand
mean(B27G0060001$meting)
```

```
[1] 337
```

```
# en de standaarddeviatie
sd(B27G0060001$meting)
```

```
[1] 57.9
```

De waardes worden niet toegewezen aan variabelen, maar we krijgen wel een beeld van de data. Het is wat omslachtig om dit allemaal apart te berekenen voor een overzicht, dus voor dat doel is de functie `summary()` handiger.

```
# descriptieve data over de grondwaterstanden
summary(B27G0060001)
```

```
   datum               meting
 Length:93          Min.   :239
 Class :character   1st Qu.:290
 Mode  :character   Median :319
                    Mean   :337
                    3rd Qu.:366
                    Max.   :482
```

Wanneer we echter willen weten wanneer de hoogste grondwaterstand bereikt werd, dan kan dat door het maximum te zoeken en vervolgens te bepalen bij welke datum dit hoort. In R nemen we dan onze variable B27G0060001 en zoeken dan in de kolom die datum heet. De vierkante haken [] gebruiken we om aan te geven waar we gaan zoeken. Zo kun je zoeken naar de eerste waarde van een kolom of lijst door bijvoorbeeld `B27G0060001$datum[1]` in te geven. We moeten daarom weten wat de positie is van de hoogste grondwaterstand in het data.frame. Daarom moeten we in de metingen zoeken naar het maximum. De code `B27G0060001$meting==max(B27G0060001$meting)` geeft aan dat we willen weten wanneer de `meting` gelijk is aan het maximum. Let op dat je de '==' gebruikt, want dat geeft aan dat je naar *is gelijk aan* wil zoeken. Bij een enkele = maak je alles gelijk aan. De code `B27G0060001$meting==max(B27G0060001$meting)` geeft een waar-onwaar matrix. Door met deze matrix te zoeken in de kolom met datum vinden we de datum waarop de hoogste grondwaterstand waar is.

```
# de eerste datum in de lijst
B27G0060001$datum[1]
```

```
[1] "1991-12-02"
```

```
# de 25e datum in de lijst
B27G0060001$datum[25]
```

```
[1] "1994-01-31"
```

```
# de meting die gelijk is aan het maximum
maximum_stand_waar <- B27G0060001$meting==max(B27G0060001$meting)
# de datum met de maximale grondwaterstand
B27G0060001$datum[maximum_stand_waar]
```

```
[1] "1994-01-31"
```

```r
# of als 1 lijn code
B27G0060001$datum[B27G0060001$meting==max(B27G0060001$meting)]
```

```
[1] "1994-01-31"
```

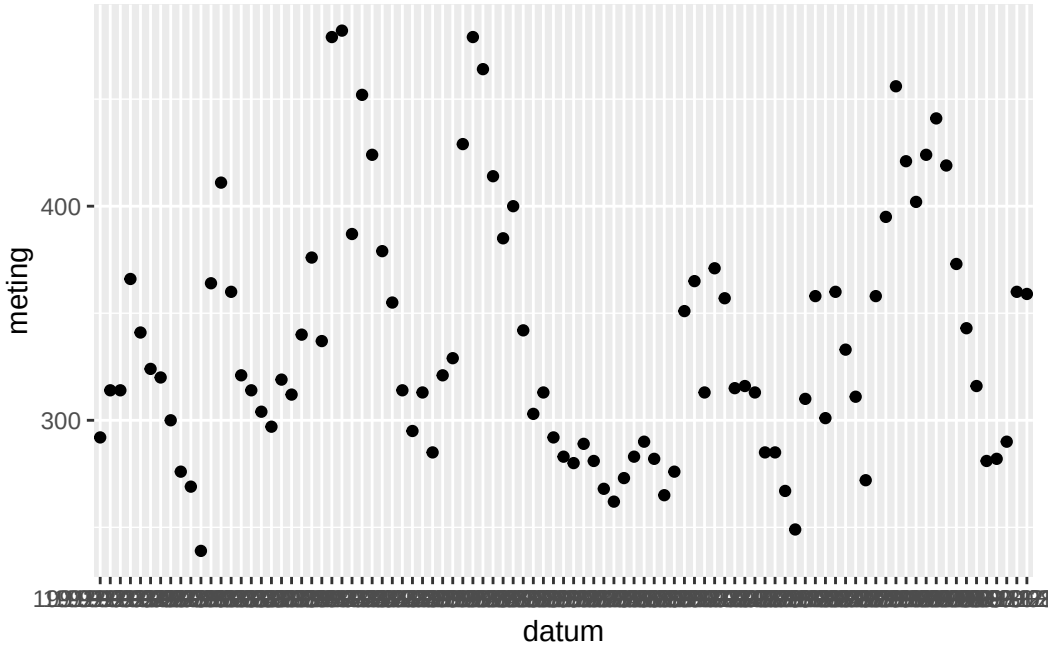
2.7 Zelf aan de slag met een aantal opdrachten

1. Zoek de datum waarop de laagste grondwaterstand werd bereikt.
2. Op welke data was de grondwaterstand lager dan de gemiddelde grondwaterstand?
3. Op welke data was de grondwaterstand lager dan de mediaan van grondwaterstand?

2.8 De eerste grafieken

We gaan nu een eerste grafiek maken met deze gegevens. Hiertoe maken we gebruik van de ggplot2 library. Daarnaast gaan we leren te werken met de syntax die hoort by de zogenaamde tidyverse. Dat doen we door gebruik te maken van een pipe (|>). Met dit sympool stuur je als het ware gegevens door. In het onderstaande voorbeeld nemen we de dataset als uitgangspunt en dan pipen we die naar de ggplot-functie, waarbij we aangeven dat de datum op de x-as komt te staan en de metingen op de y-as.

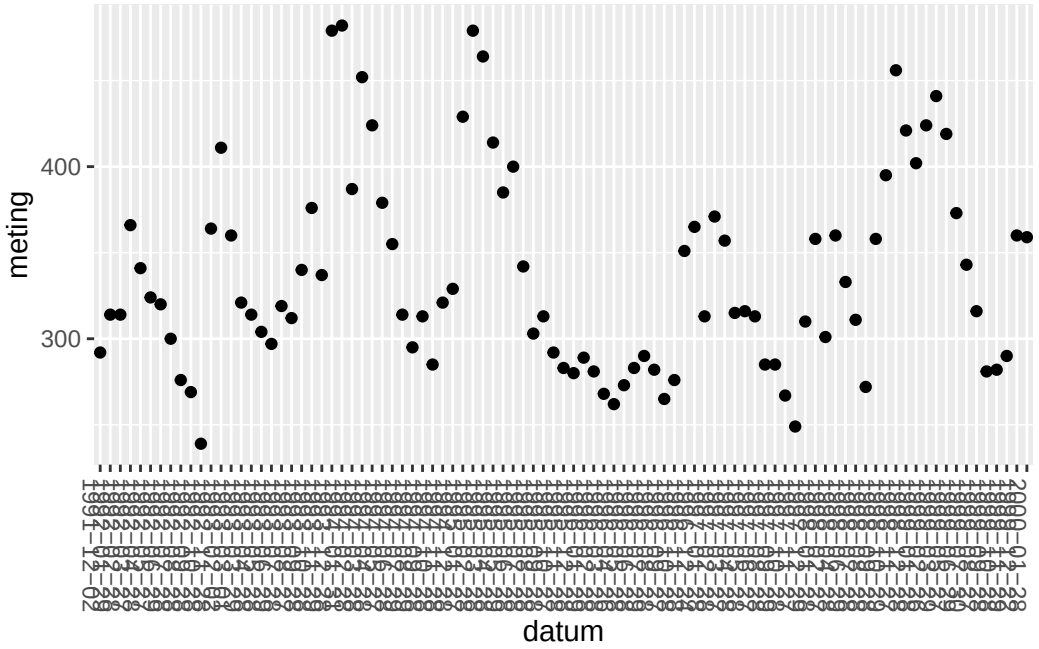
```r
# een grafiek met punten
B27G0060001 |>
  ggplot(aes(x=datum, y=meting)) + geom_point()
```



De grafiek is nog niet zo heel mooi en leesbaar. Dat ligt ook aan leesbaarheid van de x-as, dus laten we de labels roteren en er een lijn-geometrie aan toevoegen

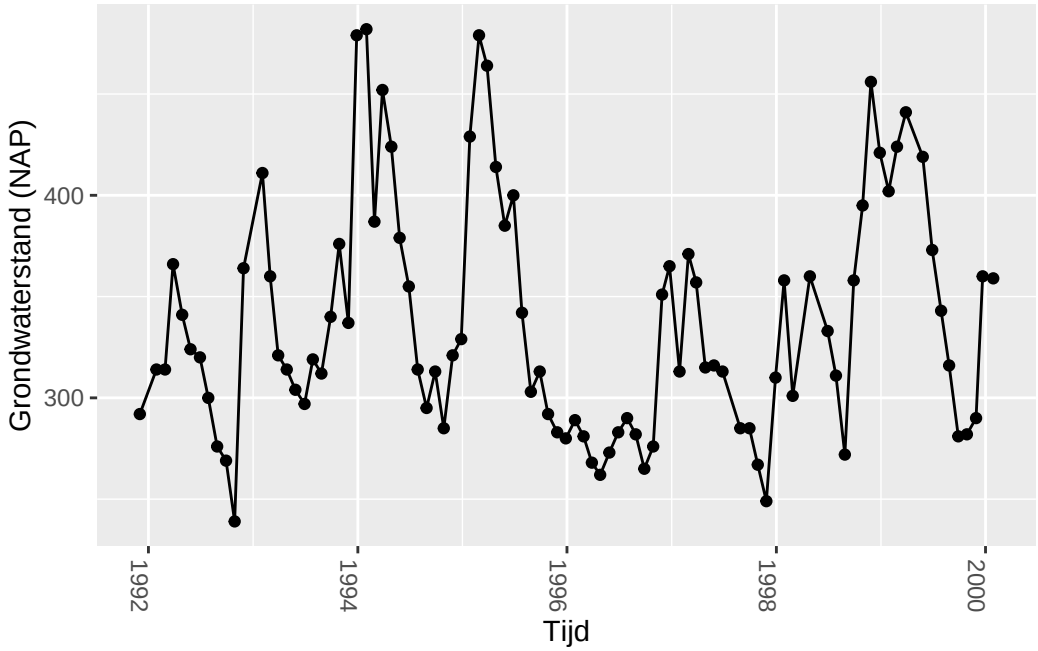
```
# een grafiek met punten
B27G0060001 |>
  ggplot(aes(x=datum, y=meting)) +
  geom_point() +
  geom_line() +
  theme(axis.text.x=element_text(angle = -90, hjust = 0))
```

```
`geom_line()`: Each group consists of only one observation.
i Do you need to adjust the group aesthetic?
```



De labels zijn nu gedraaid, maar nog niet echt leesbaar. Daarnaast worden de waardes op de x-as als nominale data ingelezen, waarbij iedere waarde uniek is, terwijl het om tijd gaat. In dit geval wordt datum als een tekst-variable gezien en niet als datum. Dat moeten we dus aangeven. Daarvoor is een standaard functie beschikbaar `as.POSIXct()`, waarbij het wel belangrijk is om aan te geven dat we de Amsterdamse tijdzone willen gebruiken. Dat kan ook in één code. Daarnaast is het ook belahgrijk om duidelijke as-labels te hebben, dus ook dat voegen we met een stukje code toe (`+ labs(x="Tijd", y="Grondwaterstand (NAP)")`).

```
# een grafiek met punten en lijn
B27G0060001 |>
  mutate(datum = as.POSIXct(datum, tz = "Europe/Amsterdam" )) |>
  ggplot(aes(x=datum, y=meting)) +
  geom_point() +
  geom_line() +
  theme(axis.text.x=element_text(angle = -90, hjust = 0)) +
  labs(x="Tijd", y="Grondwaterstand (NAP)")
```



Deze grafiek is nu beter leesbaar en de verschillende momenten in de tijd worden automatisch gegroepeerd in chronologische volgorde.

We hebben in het voorgaande voorbeeld in de code het datatype van de kolom datum tijdelijk aangepast. Dat kan ook meer blijvend bij het importeren van de data. Dat kan door de volgende code, waarmee we de kolom data muteren van een tekst-kolom naar een kolom met datum/tijd eigenschappen:

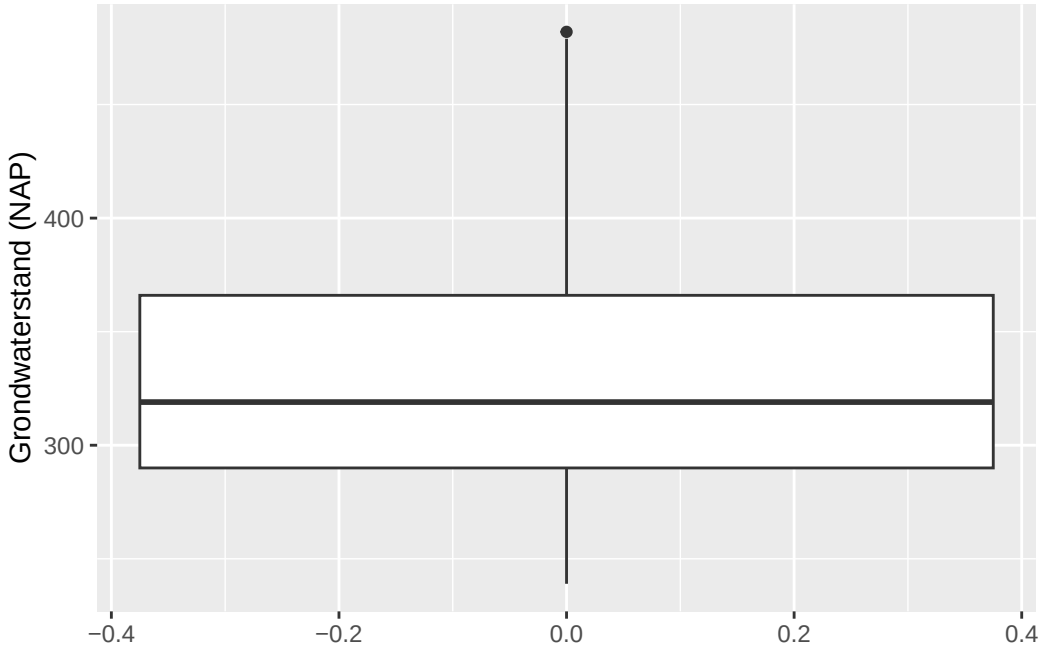
```
# tijd in data.frame goed vastleggen
B27G0060001 <- B27G0060001 |>
  mutate(datum = as.POSIXct(datum, tz = "Europe/Amsterdam" ))
```

Het is echter logischer om dit meteen bij de import van het CSV-bestand te doen. Dat kan met de onderstaande code

```
# tijd in data.frame goed vastleggen tijdens de import
B27G0060001 <- read.csv("data/B27G0060001_meetreeks.csv", sep = ";") |>
  mutate(datum = as.POSIXct(datum, tz = "Europe/Amsterdam" ))
```

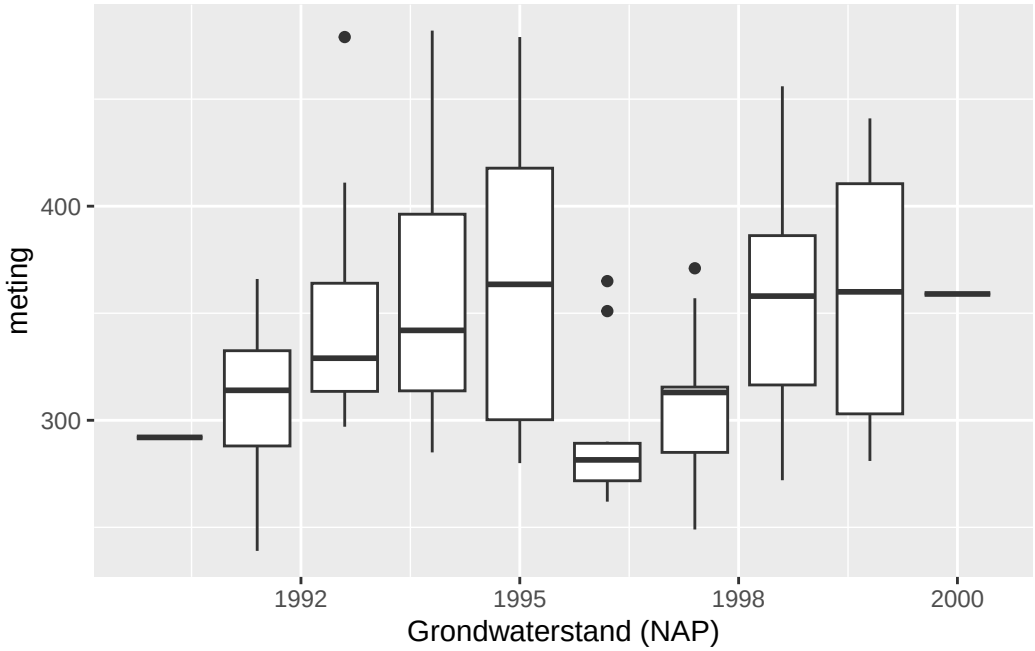
Laten we de data verder verkennen. De eerste stap is een boxplot voor alle grondwaterstanden te maken

```r
# een boxplot maken
B27G0060001 |>
  ggplot(aes(y=meting)) + geom_boxplot() +
  labs(y="Grondwaterstand (NAP)")
```



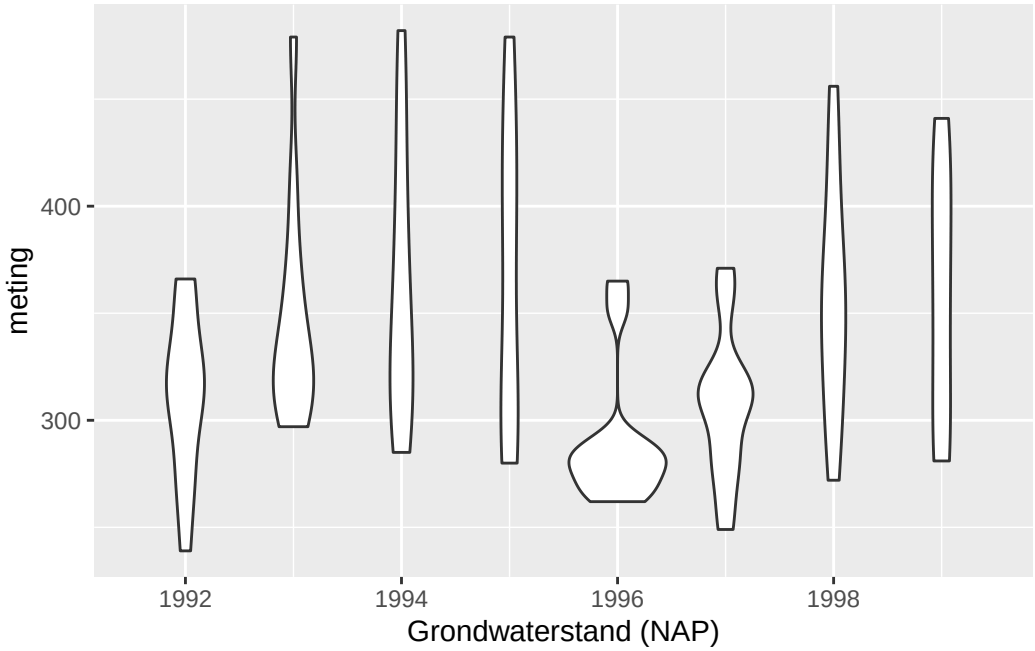
Het is echter ook mogelijk om de data te groeperen per jaar, om zo inzicht te krijgen in de waterstanden per kalenderjaar. Dit is mogelijk omdat we datum een datum-tijd veld gemaakt hebben. Dankzij het lubridate-package kunnen we makkelijk datum en tijd berekeningen maken. De functie `year()` extraheert het jaar uit een datum. In het onderstaande voorbeeld kun je dus de data groeperen op het jaar en daarna per jaar een boxplot maken.

```r
# een boxplot per jaar
B27G0060001 |>
  group_by(jaar = year(datum)) |>
  ggplot(aes(x=jaar, y = meting)) + geom_boxplot(aes(group = jaar)) +
  labs(x="Grondwaterstand (NAP)")
```



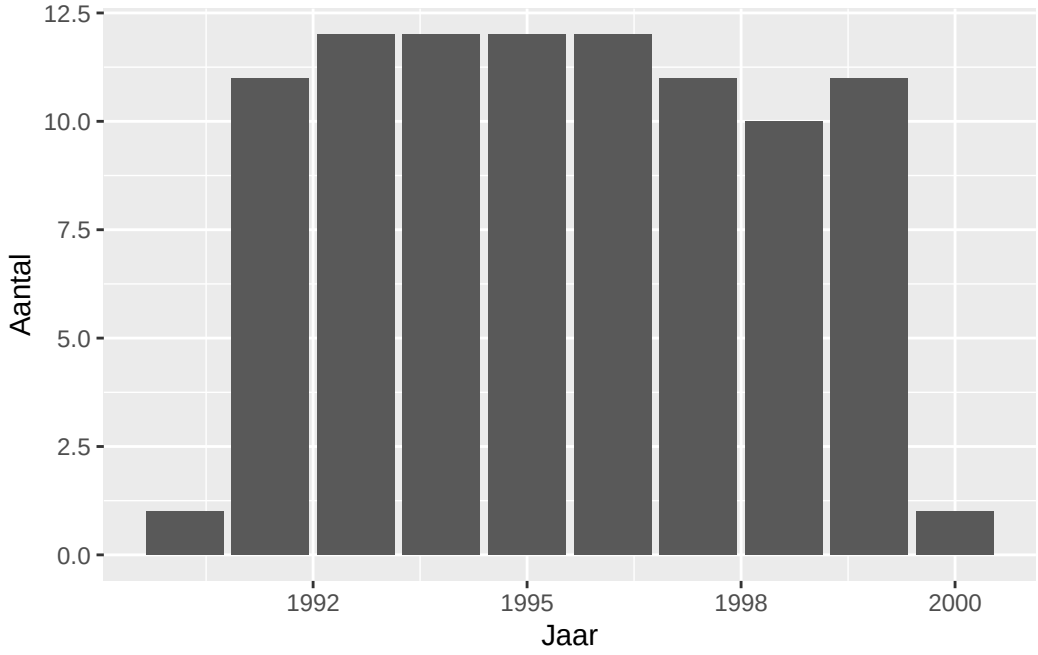
Het valt nu op dat er veel variatie zit in de jaren, maar ook dat in sommige jaren de waardes wat dichter bij elkaar liggen. Dit is moeilijk uit een boxplot te halen, maar een *violin plot* geeft soms meer inzicht in hoe de waardes verdeeld zijn.

```
# een violin plot per jaar
B27G0060001 |>
  group_by(jaar = year(datum)) |>
  ggplot(aes(x=jaar, y = meting)) + geom_violin(aes(group = jaar)) +
  labs(x="Grondwaterstand (NAP)")
```



Het is dan ook belangrijk om te weten hoeveel metingen er per jaar zijn. Dat is eenvoudig te visualiseren met behulp van een staafdiagram. Hiertoe moeten we eerst de data weer groeperen per jaar en daarna tellen hoeveel metingen er per jaar zijn. Standaard gaat ggplot uit van het zelf bepalen van de aantallen op basis van de data, maar in dit geval weten we wat de x en de y moet worden en vertellen we dat aan `geom_bar()` door `geom_bar(stat = "identity")` als argument mee te geven, hiermee kun je zelf de x en de y vaststellen.

```
# een bar plot voor aantal metingen per jaar
B27G0060001 |>
  group_by(jaar = year(datum)) |>
  summarise(aantal = n()) |>
  ggplot(aes(x = jaar, y = aantal)) + geom_bar(stat = "identity") +
  labs(x="Jaar", y="Aantal")
```



Tip

Wil je meer weten over het visualiseren van tijdreeksen, staan hier wat handige tips: https://r-graph-gallery.com/279-plotting-time-series-with-ggplot2.html

2.9 Zelf aan de slag met een aantal opdrachten

1. Maak de bovenstaande grafieken (boxplot en staafdiagram van aantal metingen per jaar) ook maken voor de maanden. Op die manier kun je zien of er in bepaalde maanden vaker de grondwaterstanden vaker gemeten is.
2. Het is ook mogelijk om een trend te visualiseren in `ggplot()`. Hiertoe gebruik je `geom_smooth()`. Maak een grafiek met de boxplots per jaar, met daaraan toegevoegd de trendlijn
3. Maak een boxplot, waarbij je laat zien wat de grondwaterstanden zijn voor ieder van de vier seizoenen. (https://www.knmi.nl/over-het-knmi/nieuws/vier-of-zes-seizoenen)

2.10 Op zoek naar aanvullende data

Het is misschien opgevallen dat de metingen na 2000 ophouden. Voor de meer recente data hebben we blijkbaar een andere bron nodig. Gelukkig kan dat via https://opendata.munisense.

net/nl/. Daar vinden we een punt dat zeer vlakbij het meetpunt uit de voorgaande oefening ligt: meetpunt 660721.

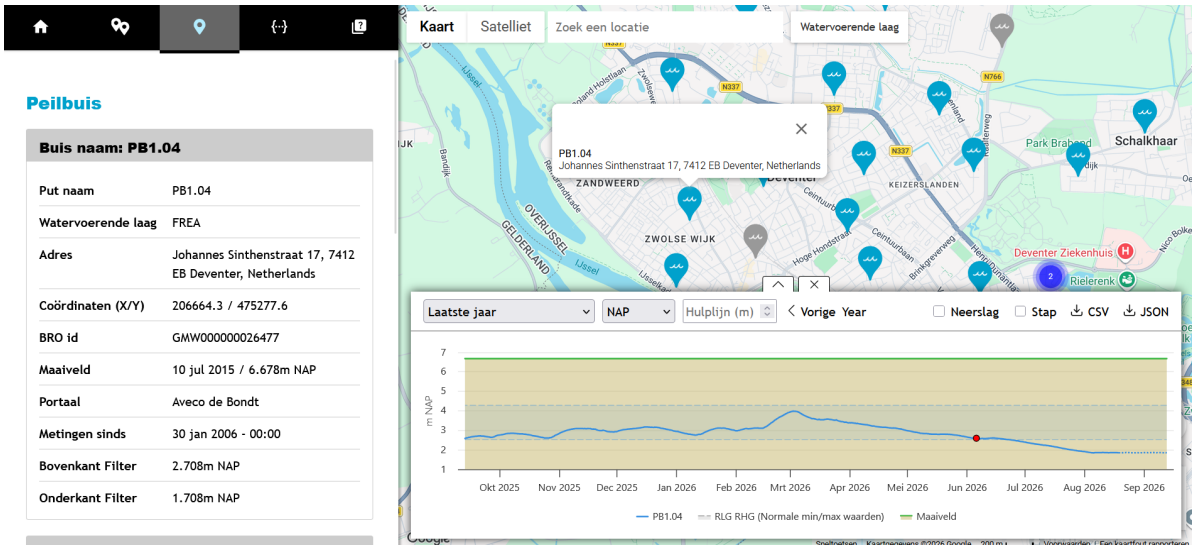
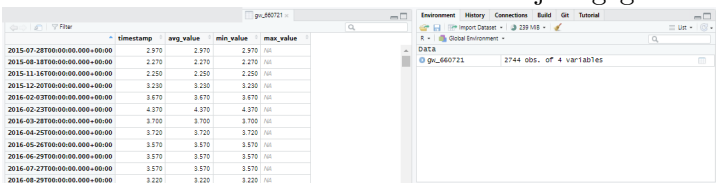


Figure 2.4: De website van https://opendata.munisense.net/nl/

Wanneer je een CSV-bestand met alle metingen van het meetpunt download en opent in een tekst-editor, zul je zien dat de eerste twee regels geen data bevatten. Die kun je negeren door `skip = 2` aan de opdracht mee te geven.

```
# tijd in data.frame goed vastleggen
gw_660721 <- read.csv("data/660721-water_level_filtered-all.csv", skip = 2)
```

Als je nu vanuit 'Environment' de variable bekijkt (klik hier voor een uitleg), zul je zien dat de eerste kolom naar de naam van de rij is gegaan en de laatste kolom leeg lijkt te zijn.  Deze CSV is blijkbaar niet netjes opgemaakt en wanneer je deze in een teksteditor opent, valt het volgende op: - De kolomkoppen zijn als volgt: timestamp,avg_value,min_value,max_value - De regels met data zijn als volgt: "2015-07-28T00:00:00.000+00:00","2.97","2.97","2.97",

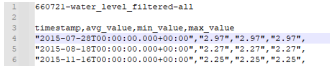


Figure 2.5: De CSV van 660721 in een tekst editor

Omdat de datavelden eindigen met een komma, wordt er verwacht dat er nog een kolom met data is, maar de kolomkoppen hebben dat niet. Dan leest R de data wel in. De makkelijkste oplossing is nu een komma extra te plaatsen na de kolomkoppen. Dat ziet er dan zo uit.

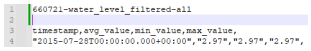


Figure 2.6: De gecorrigeerde CSV van 660721 in een tekst editor

Deze slaan we op als een nieuw bestand, zodat we altijd de originele data behouden. Op deze manier kun je de data inlezen zonder problemen. Het nadeel is wel dat er een extra lege kolom wordt aangemaakt, maar die kun je weer weghalen, door de andere kolommen te selecteren Daarnaast is het handig om meteen het timestamp veld als datum/tijd in te lezen.

```
# tijd in data.frame goed vastleggen
gw_660721 <- read.csv("data/660721-water_level_filtered-all_edit.csv", skip = 2) |>
  select(timestamp:max_value) |>
   mutate(timestamp = as.POSIXct(timestamp, tz = "Europe/Amsterdam" ))
```

Dit werkt voor een enkel bestand, maar wanneer je veel grondwaterstanden van Munisense moet inlezen, is dat erg bewerkelijk. Je zou ook de data in kunnen lezen en de regel met koppen kunnen overslaan, om daarna de veldnamen te wijzigen. De velden hebben echter geen goede naam, dus om niet de lege kolom te selecteren, selecteren we eerst kolommen 1 tot en met 4, delen daarna namen uit en veranderen het timestamp veld in een datum/tijd veld.

```
# tijd in data.frame goed vastleggen
gw_660721 <- read.csv("data/660721-water_level_filtered-all.csv", skip = 3) |>
  select(1:4) |>
  `colnames<-`(c("timestamp","avg_value","min_value","max_value")) |>
  mutate(timestamp = as.POSIXct(timestamp, tz = "Europe/Amsterdam" ))
```

> **Tip**
>
> De Wet Basisregistratie Ondergrond maakt het mogelijk dat alle publieke gegevens van de Nederlandse ondergrond toegankelijk via één loket. Dat is het BROloket en daar vinden we ook al deze data: https://www.broloket.nl/ondergrondgegevens

2.11 Zelf aan de slag met een aantal opdrachten

1. Zoek de datum waarop de laagste grondwaterstand werd bereikt.
2. Op welke data was de grondwaterstand lager dan de gemiddelde grondwaterstand?

3. Op welke data was de grondwaterstand lager dan de mediaan van grondwaterstand?
4. Maak een lijndiagram met de grondwaterstanden van deze dataset, inclusief trendlijn.
5. Maak een staafdiagram met het aantal metingen per jaar.
6. Maak een viool-diagram, zoals eerder gemaakt, maar nu met deze dataset.

2.12 Combineren van datasets

We hebben nu twee datasets met grondwaterstanden van meetpunten die vlak bij elkaar liggen. Laten we deze eens combineren, want op die manier kunnen we voor een langere periode de grondwaterstanden zien.

Om datasets te combineren is praktisch om dezelde veldnamen te gebruiken. Het commando colnames is handig om namen op te vragen, maar ook om ze toe te kennen. Laten we ze eerst eens bekijken:

```
# veldnamen
colnames(B27G0060001)
```

```
[1] "datum"  "meting"
```

```
colnames(gw_660721)
```

```
[1] "timestamp" "avg_value" "min_value" "max_value"
```

Hoewel`gw_660721` vier kolommen heeft, zijn de data in de laatste drie kolommen identiek. Beide datasets hebben een veld met datum en een veld met een grondwaterstand. Als het goed is, valt het ook op dat de eenheid in de ene dataset in meters is, en in de andere in centimeters. Dat moet ook gelijk gemaakt worden. De volgende stappen moeten worden doorlopen - Uniformeren kolomnamen (gebruik maken van `mutate()`) - Uniformeren eenheid getallen (gebruik maken van `mutate()`) - Bron herleidbaar maken (kolom toevoegen met `mutate()`) - Samenvoegen dataset (`rbind()`, zie hier voor meer voorbeelden)

```
# omzetten datasets
grondwaterstanden <- rbind(
  B27G0060001 |>
    mutate(grondwaterstand = meting/100) |>
    mutate(bron = "grondwatertools") |>
    select(datum, grondwaterstand, bron),
  gw_660721 |>
    mutate(grondwaterstand = avg_value) |>
```

```
    mutate(datum = timestamp) |>
    mutate(bron = "munisense") |>
    select(datum, grondwaterstand, bron)
)
```

Dit kan dus met één samengestelde opdracht.

2.13 Zelf aan de slag met een aantal opdrachten

1. Zoek de datum waarop de laagste grondwaterstand werd bereikt.
2. Op welke data was de grondwaterstand lager dan de gemiddelde grondwaterstand?
3. Op welke data was de grondwaterstand lager dan de mediaan van grondwaterstand?
4. Maak een lijndiagram met de grondwaterstanden van deze dataset, inclusief trendlijn.
5. Maak een staafdiagram met het aantal metingen per jaar.
6. Maak een viool-diagram, zoals eerder gemaakt, maar nu met deze dataset.

Wat valt op aan de datasets? Welke data mis je nog?

2.14 Huiswerk: verwerken in de eigen projecten

Ga op zoek naar de verschillende meetpunten van grondwaterstanden in jouw onderzoeksgebied. Probeer inzicht te krijgen in het verloop van deze grondwaterstanden. Welke rol speelt grondwater in de lokale problemen in jullie onderzoeksgebied?

3 Werkcollege 2: werken met weer- en klimaatdata

3.1 Inleiding

In het vorige werkcollege hebben jullie gewerkt met data van grondwaterstanden uit verschillende bronnen. Daarnaast hebben jullie diverse grafieken gemaakt. Deze week gaan we aan de slag met gegevens over het weer en klimaat.

3.2 Weerdata

Data over het weer wordt door het KNMI als open data aangeboden. Dit is tegenwoordig via https://dataplatform.knmi.nl/ toegankelijk. De data is toegankelijk via een API. Dit maakt de toegank voor een deel makkelijker, maar er is wel wat code voor nodig. Gelukkig biedt het KNMI ook data aan als eenvoudige downloads: https://www.knmi.nl/nederland-nu/klimatologie/uurgegevens. Hierover wordt veel uitgelegd op de website.

We gaan werken met de data uit een bepaald gebied. Laten we eens naar het zuiden van het land kijken en in Limburg op zoek gaan naar data. Op https://www.knmi.nl/kennis-en-datacentrum/uitleg/automatische-weerstations staat een kaartje met de locaties van alle weerstations. In Limburg is een een weerstation bij Maastricht Airport (380).

Ga naar https://www.knmi.nl/nederland-nu/klimatologie/uurgegevens en download de meest recente data uit Maastricht (2021-2030). Dit bestand zal `uurgeg_380_2021-2030.txt` heten.

3.2.1 Project en script aanmaken

Je hebt vorig week geleerd een project aan te maken. Maak een nieuw project aan in een nieuwe map. Deze noemen we nu Grounded_WC2.

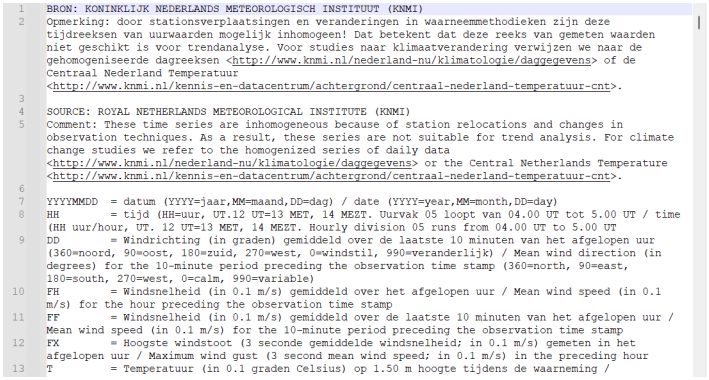
Maak ook een script aan en geef deze een logische en passend naam. We beginnen weer met het aanroepen van de juiste bibliotheken. Dat zijn deze keer wederom dplyr (Wickham, François, et al. 2023), ggplot (Wickham 2016) en lubridate (Grolemund and Wickham 2011). Daarnaast gaan we nieuwe libraries gebruiken: tidyr (Wickham, Vaughan, et al. 2023), ggpubr (Kassambara 2023) en readr (Wickham et al. 2026).

```r
# load libraries
library(dplyr)
library(ggplot2)
library(lubridate)
library(tidyr)
library(ggpubr)
library(readr)

# force non scientific formatting of numbers
options(scipen = 999)
options(digits=3)
```

3.2.2 Data inlezen en de eerste verkenning van de data

Bekijk de data die je gedownload hebt in een tekst-editor. Je zult zien dat de data bestaat uit een hoop uitleg. Dat is heel fijn, omdat je meteen weet wat de data beteken.



```
BRON: KONINKLIJK NEDERLANDS METEOROLOGISCH INSTITUUT (KNMI)
Opmerking: door stationsverplaatsingen en veranderingen in waarneemmethodieken zijn deze
tijdreeksen van uurwaarden mogelijk inhomogeen! Dat betekent dat deze reeks van gemeten waarden
niet geschikt is voor trendanalyse. Voor studies naar klimaatverandering verwijzen we naar de
gehomogeniseerde dagreeksen <http://www.knmi.nl/nederland-nu/klimatologie/daggegevens> of de
Centraal Nederland Temperatuur
<http://www.knmi.nl/kennis-en-datacentrum/achtergrond/centraal-nederland-temperatuur-cnt>.

SOURCE: ROYAL NETHERLANDS METEOROLOGICAL INSTITUTE (KNMI)
Comment: These time series are inhomogeneous because of station relocations and changes in
observation techniques. As a result, these series are not suitable for trend analysis. For climate
change studies we refer to the homogenized series of daily data
<http://www.knmi.nl/nederland-nu/klimatologie/daggegevens> or the Central Netherlands Temperature
<http://www.knmi.nl/kennis-en-datacentrum/achtergrond/centraal-nederland-temperatuur-cnt>.

YYYYMMDD  = datum (YYYY=jaar,MM=maand,DD=dag) / date (YYYY=year,MM=month,DD=day)
HH        = tijd (HH=uur, UT.12 UT=13 MET, 14 MEZT. Uurvak 05 loopt van 04.00 UT tot 5.00 UT / time
(HH uur/hour, UT. 12 UT=13 MET, 14 MEZT. Hourly division 05 runs from 04.00 UT to 5.00 UT
DD        = Windrichting (in graden) gemiddeld over de laatste 10 minuten van het afgelopen uur
(360=noord, 90=oost, 180=zuid, 270=west, 0=windstil, 990=veranderlijk) / Mean wind direction (in
degrees) for the 10-minute period preceding the observation time stamp (360=north, 90=east,
180=south, 270=west, 0=calm, 990=variable)
FH        = Windsnelheid (in 0.1 m/s) gemiddeld over het afgelopen uur / Mean wind speed (in 0.1
m/s) for the hour preceding the observation time stamp
FF        = Windsnelheid (in 0.1 m/s) gemiddeld over de laatste 10 minuten van het afgelopen uur /
Mean wind speed (in 0.1 m/s) for the 10-minute period preceding the observation time stamp
FX        = Hoogste windstoot (3 seconde gemiddelde windsnelheid; in 0.1 m/s) gemeten in het
afgelopen uur / Maximum wind gust (3 second mean wind speed; in 0.1 m/s) in the preceding hour
T         = Temperatuur (in 0.1 graden Celsius) op 1.50 m hoogte tijdens de waarneming /
```

Figure 3.1: Data van het KNMI (uurgegevens)

Om de data in te kunnen lezen, plaatsen we het bestand in de `data`-map. Deze had je uiteraard al aangemaakt op basis van de les van vorige week (dat kan met code of middels bestandsbeheer).

Als we de data in proberen te lezen, zal dit niet helemaal goed gaan met het onderstaande commando.

````
```{r}
maastricht_2021_2030 <- read.csv("data/uurgeg_380_2021-2030.txt")
```
````

Je zult het bestand dus even goed moeten bekijken. Dan zul je erachter komen dat je een aantal regels moet overslaan bij het importeren, namelijk 31 regels.

```
maastricht_2021_2030 <- read.csv("data/uurgeg_380_2021-2030.txt", skip = 31)
```

Laten we de data even bekijken. Hieronder staan de eerste 10 regels uit de ingelezen tabel.

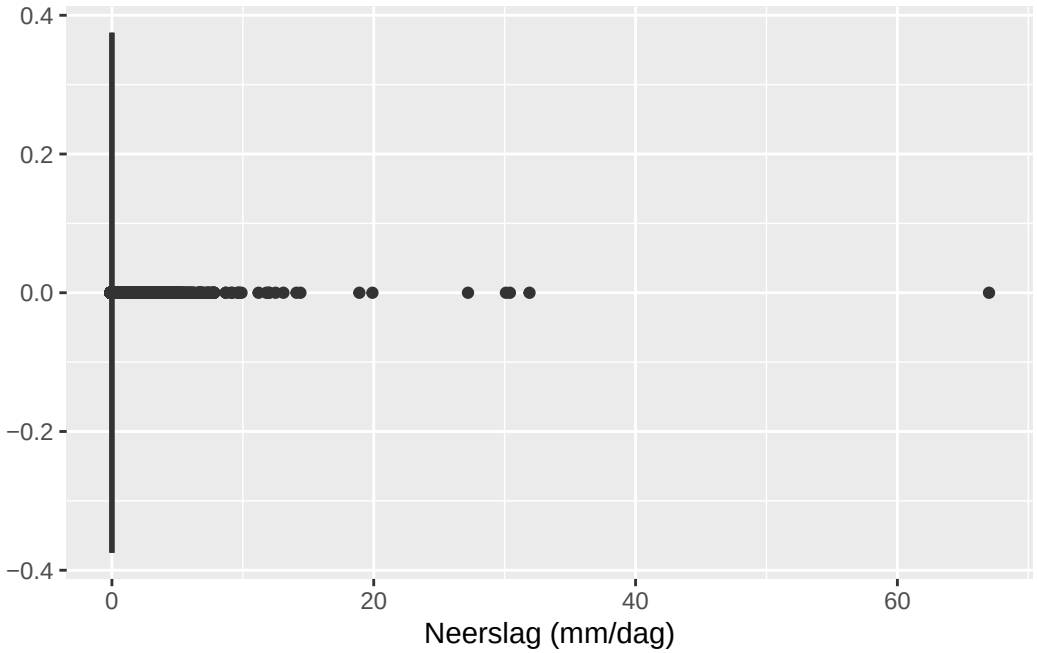
X..STN	YYYYMMDD	HH	DD	FH	FF	FX	T	T10N	TD	SQ	Q	DR	RH	P	VV	N	U	WW	IX	M	R	S	O	Y
380	20210101	1	310	20	20	30	21	NA	16	0	0	0	0	10061	23	8	96	NA	NA	0	0	0	0	0
380	20210101	2	320	20	10	30	20	NA	15	0	0	0	0	10067	12	8	96	NA	NA	0	0	0	0	0
380	20210101	3	340	10	10	30	19	NA	10	0	0	0	-1	10075	50	8	94	NA	NA	0	1	0	0	0
380	20210101	4	250	10	10	10	17	NA	10	0	0	0	0	10071	28	8	95	NA	NA	0	0	0	0	0
380	20210101	5	230	10	10	20	15	NA	10	0	0	0	-1	10073	22	8	96	NA	NA	0	1	0	0	0
380	20210101	6	240	10	20	30	14	10	7	0	0	0	-1	10077	36	8	95	NA	NA	0	1	0	0	0
380	20210101	7	220	20	20	30	10	NA	4	0	0	0	-1	10084	31	8	96	NA	NA	0	1	0	0	0
380	20210101	8	210	20	20	40	14	NA	7	0	1	0	0	10087	26	8	95	NA	NA	0	0	0	0	0
380	20210101	9	220	20	20	30	18	NA	9	2	14	0	0	10093	30	8	94	NA	NA	0	0	0	0	0
380	20210101	10	220	30	30	40	25	NA	14	0	27	0	0	10096	50	8	92	NA	NA	0	0	0	0	0

Het valt op dat in de kolom RH negatieve waarde voorkomen. In het ingelezen tekstbestand lezen we:

```
RH         = Uursom van de neerslag (in 0.1 mm) (-1 voor <0.05 mm)
```

Dat betekent dus dat negatieve waardes wel neerslag betekenen, maar minimaal. Wanneer we dit plotten in een boxplot zie je meteen dat er veel lage waardes zijn.

```
maastricht_2021_2030 |>
  ggplot(aes(x=(RH/10))) + geom_boxplot() +  labs(x = "Neerslag (mm/dag)")
```



In bovenstaande code is de neerslag getransformeerd naar millimeter per dag, omdat de meeste mensen deze eenheid herkenbaarder zullen vinden dan een neerslag in 0.1 mm.

3.2.2.1 Nu zelf

1. Wat is de gemiddelde neerslag per uur? Welke standaarddeviatie hoort daarbij?
2. Wat is de modale neerslag per uur?
3. Maak een histogram van de neerslag per dag (in mm). Zorg daarbij dat de assen ook een duidelijke benaming krijgen.

Laten we even alle negatieve waardes omzetten naar een waarde die niets betekent, oftewel `NA`. Dat betekent dat we geen waardes hebben voor die momenten. Hiertoe zoeken we alle plekken op waar een -1 staat en vervangen dat voor `NA`:

```
maastricht_2021_2030[maastricht_2021_2030==-1] <- NA
```

3.2.2.2 Nu zelf

Als je NA in de data hebt, moet je vaak `, na.rm = TRUE` aan een functie toevoegen. Kijk maar eens in de help van `mean()`.

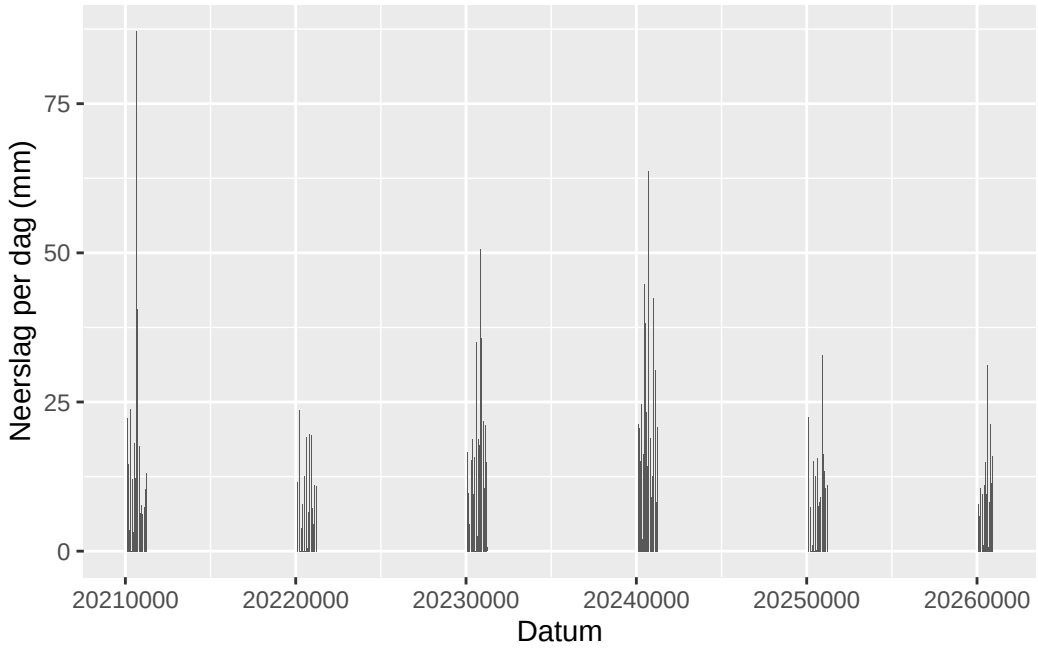
1. Wat is de gemiddelde neerslag per uur? Welke standaarddeviatie hoort daarbij?

2. Wat is de modale neerslag per uur?
3. Op welke dag viel de meeste neerlag per uur?
4. Maak een nieuwe boxplot van de neerslag. Zie je een verschil met die we eerder hebben gemaakt zonder de -1 te verwijderen?
5. Maak een nieuw histogram van de neerslag per dag (in mm). Zorg daarbij dat de assen ook een duidelijke benaming krijgen.

3.3 Grafieken maken voor de neerslag per dag

De neerslag per uur geeft al veel informatie, maar soms wil je eerder weten wat de hoeveelheden neerslag per dag was. Dat kan vrij eenvoudig door data te groeperen en de som per groep te berekenen. Laten we dat eens doen.

```
maastricht_2021_2030 |>
  group_by(YYYYMMDD) |>
  summarise(neerslagsom = sum((RH/10), na.rm = TRUE)) |>
  ggplot(aes(x=YYYYMMDD, y = neerslagsom)) + geom_bar(stat = "Identity") +
  labs(x = "Datum", y = "Neerslag per dag (mm)")
```



Er val nu meteen op dat er een aantal pieken is, en dat die vrij ver uit elkaar liggen. Dat komt doordat het datatype van het `YYYYMMDD`-veld een integer is. Dit kun je opzoeken in R door de functie `class()` te gebruiken.

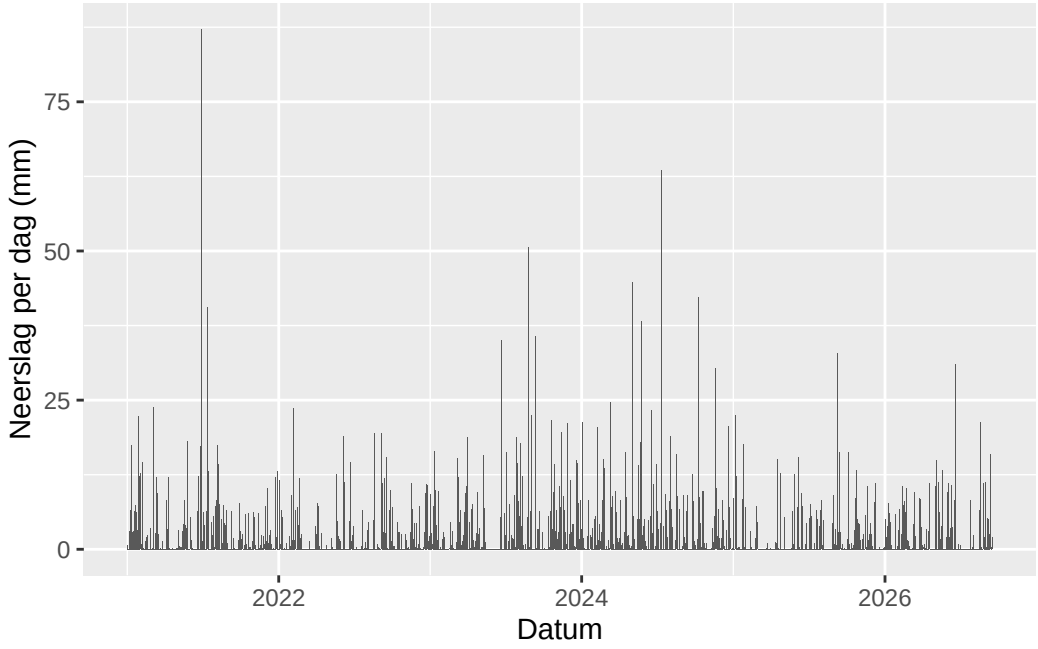
```
class(maastricht_2021_2030$YYYYMMDD)
```

```
[1] "integer"
```

> **Tip**
>
> Wil je weten wat voor gegevenstype een variabele of een bepaalde kolom in een data.frame heeft, gebruik je de functie `class()`.

In het vorige werkcollege heb je al gewerkt met de functies uit de `lubridate`-package (Grolemund and Wickham 2011). Aangezien het veld bestaat uit jaar-maand-dag, kunnen we de functie `ymd()` daarvoor gebruiken en een nieuw veld aan te maken dat we datum noemen.

```
maastricht_2021_2030 |>
  mutate(datum = ymd(YYYYMMDD)) |>
  group_by(datum) |>
  summarise(neerslagsom = sum(RH/10, na.rm = TRUE)) |>
  ggplot(aes(x=datum, y = neerslagsom)) + geom_bar(stat = "Identity") +
    labs(x = "Datum", y = "Neerslag per dag (mm)")
```



Op deze manier wordt de grafiek veel leesbaarder en lijkt ook de variatie beter inzichtelijk te zien. In het bovenstaande voorbeeld hebben we de datum aangepast bij het maken van

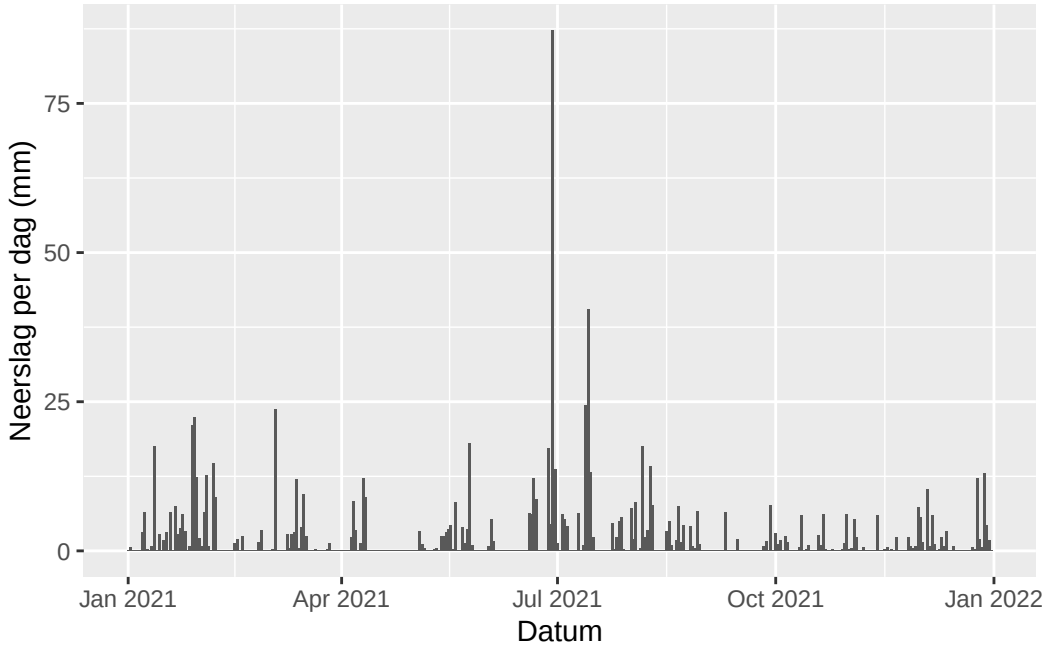
de grafiek. Dat is voor een enkele keer handig, maar eigenlijk wat omslachtig. Dit betekent namelijk dat je dit bij iedere bewerking opnieuw moet bepalen, terwijl het handiger is om dit soort dingen in één keer aan te passen, zodat je daarna kunt werken met de bewerkte gegevens. Dat moet je dus eigenlijk meteen doen bij het inlezen van jouw bestand, zodat je daarna alle bewerkingen kunt doen, zonder steeds opnieuw de data te corrigeren. Dat geldt natuurlijk ook voor de neerslag in mm, dus dat passen we ook aan.

```
maastricht_2021_2030 <- read.csv("data/uurgeg_380_2021-2030.txt", skip = 31, colClasses = "in
  mutate_all(~na_if(., -1)) |>
  mutate(datum = ymd(YYYYMMDD)) |>
  mutate(neerslag_mm = RH/10)
```

In de bovenstaande code lezen we eerst de CSV in door `maastricht_2021_2030 <- read.csv("data/uurgeg_380_2021-2030.txt", skip = 31, colClasses = "integer")`. In dit voorbeeld zorgen we dat alle kolommen als een integer worden ingelezen, omdat deze data als zodanig wordt aangeleverd door het KNMI. Daarna vervangen we alle -1 waardes voor een `NA` door `mutate_all(~na_if(., -1))`. De laatst twee regels voegen een datum-veld en veld met de neerslag in millimeter.

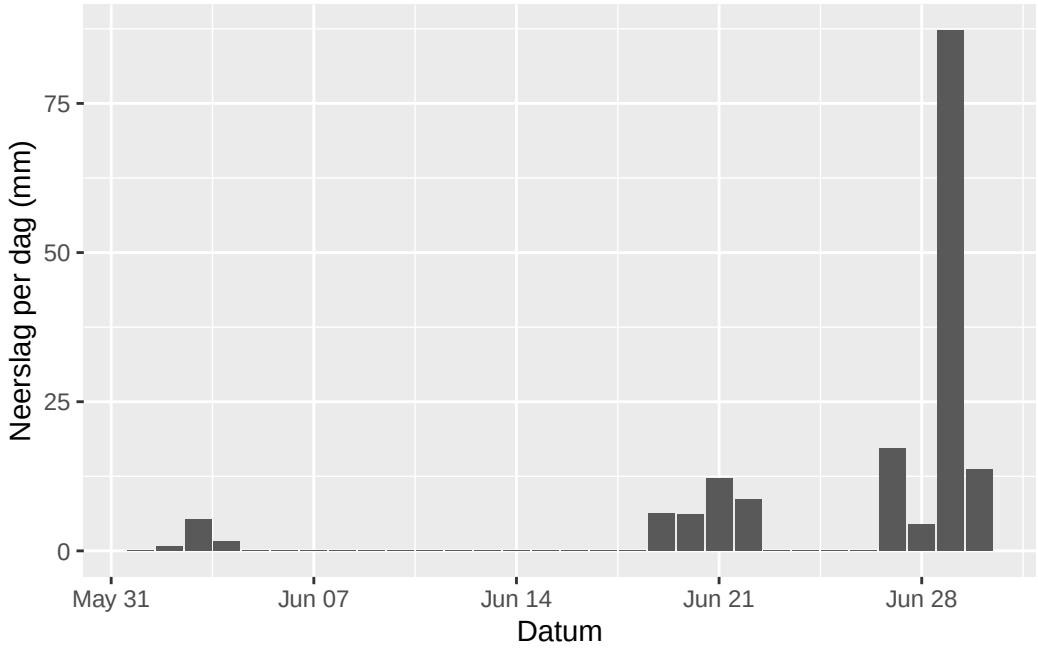
Terug naar de grafiek. Als het goed is, is het je opgevallen dat er een enorme piek ergens voor 2022 zichtbaar is. Laten we eens inzoomen op de grafiek in de periode voor 2022, door alleen de data voor 2022 te selecteren:

```
maastricht_2021_2030 |>
  group_by(datum) |>
  filter(YYYYMMDD<20220000) |>
  summarise(neerslagsom = sum(neerslag_mm, na.rm = TRUE)) |>
  ggplot(aes(x=datum, y = neerslagsom)) + geom_bar(stat = "Identity") +
    labs(x = "Datum", y = "Neerslag per dag (mm)")
```



We zien nu dat dit ergens in juni moet zijn geweest, dus laten we eens wat meer in detail kijken naar de maand juni van 2022. Dat kan met de onderstaande code

```
maastricht_2021_2030 |>
  group_by(datum) |>
  filter(YYYYMMDD >= 20210600 & YYYYMMDD <= 20210700) |>
  summarise(neerslagsom = sum(neerslag_mm, na.rm = TRUE)) |>
  ggplot(aes(x=datum, y = neerslagsom)) + geom_bar(stat = "Identity") +
    labs(x = "Datum", y = "Neerslag per dag (mm)")
```



3.3.1 Weer zelf aan de slag

1. Op welke datum valt deze piek van regen?
2. Maak een staafdiagram van de regen op deze dag, waarbij je per uur aangeeft hoe veel regen er gevallen is. Hoe laat valt de piek?
3. Wat kun je op het nieuws vinden over deze neerslagpiek? Welke gevolgen had dit voor mensen?
4. Welke grondwaterpunten zijn er in de buurt van de meetlocatie te vinden?
5. Maak voor deze meetlocatie een grafiek van de grondwaterstanden. Is er een relatie te vinden tussen neerslag en grondwaterstanden?

3.4 Temperatuur

We hebben nu een aantal grafieken gemaakt voor de neerslag, maar neerslag is niet de enige weerfactor die een bepaalde rol speelt. Er zijn er vele, zoals wind, intensiteit van de zon. Voor het overzicht groeperen we ze hieronder. Ieder heeft andere invloed op de leefomgeving. Dat betreffen:

- Wind (-richting, -snelheid, -stoot)
- Temperatuur (meting, gemiddeld, dauwpunt)
- Zon (duur, straling)

- Neerslag (duur per uur, hoeveelheid)
- Luchtdruk
- Zicht
- Bewolking
- Weersomstandigheden, zoals mist, sneeuw en onweer

Het voert te ver om al deze gegevens te gebruiken in dit werkcollege, maar je zou ze voor verschillende zaken kunnen gebruiken, zoals het bepalen van hittestress of onderzoeken welke invloed mist heeft op de hoeveelheid ongelukken. Dergelijke zaken kun je koppelen aan de inrichting van de leefomgeving en zo tot andere inzichten komen.

Laten we eens verder gaan met de temperatuur in beeld te brengen.

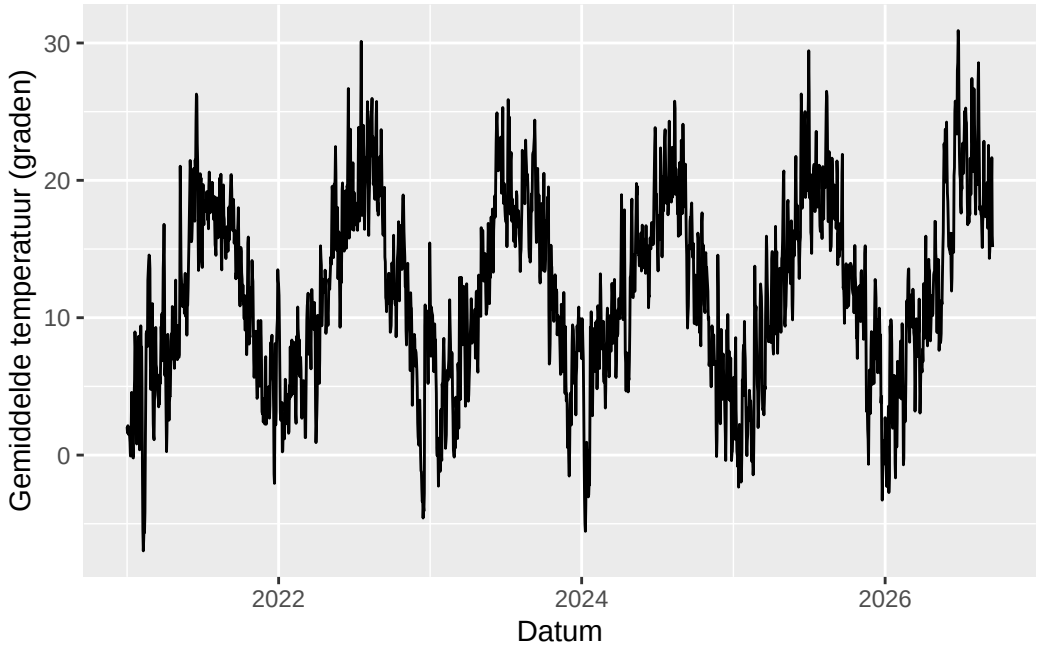
```
T         = Temperatuur (in 0.1 graden Celsius) op 1.50 m hoogte tijdens de
waarneming / Temperature (in 0.1 degrees Celsius) at 1.50 m at the time of
observation
```

Ook deze moeten we dus weer aanpassen naar een eenheden in graden en hadden we aan het begin bij importeren natuurlijk moeten bepalen

```
maastricht_2021_2030 <- read.csv("data/uurgeg_380_2021-2030.txt", skip = 31, colClasses = "i
  mutate_all(~na_if(., -1)) |>
  mutate(datum = ymd(YYYYMMDD)) |>
  mutate(neerslag_mm = RH/10) |>
  mutate(temperatuur_graden = T/10)
```

Laten we eerst eens kijken hoe de gemiddelde temperatuur voor iedere dag in het jaar verloopt.

```
maastricht_2021_2030 |>
  group_by(datum) |>
  summarise(gem_temp = mean(temperatuur_graden, na.rm = TRUE)) |>
  ggplot(aes(x=datum, y = gem_temp)) + geom_line() + labs(x="Datum", y = "Gemiddelde temperat
```



Hierin is duidelijk de trend te zien van de seizoenen. De bovenstaande grafiek geeft echter alleen de gemiddelde temperatuur, maar niet de laagste of de hoogste per dag. Dat is allemaal samen te berekenen en in een grafiek te zetten. Hiertoe moeten we eerst de data samenvatten (`summarise`) en de gemiddeldes, minima en maxima per dag te berekenen. Dan worden er aparte kolommen aangemaakt per waarde. Als we dit echter in een grafiek willen zetten en de kleuren automatisch willen maken, moeten we de kolommen omzetten naar het zogenaamde *longer*-formaat. Het commando `pivot_longer(cols = c(gem_temp, max_temp, min_temp))` zorgt ervoor dat de kolom datum behouden blijft en dat in het veld `name` de titel van de kolommen gem_temp, max_temp en min_tempkomt te staan. In het veld `value` komen de waardes te staan. Je hebt in feite de verschillende gegevens getransporteerd en naar een ander formaat omgezet:

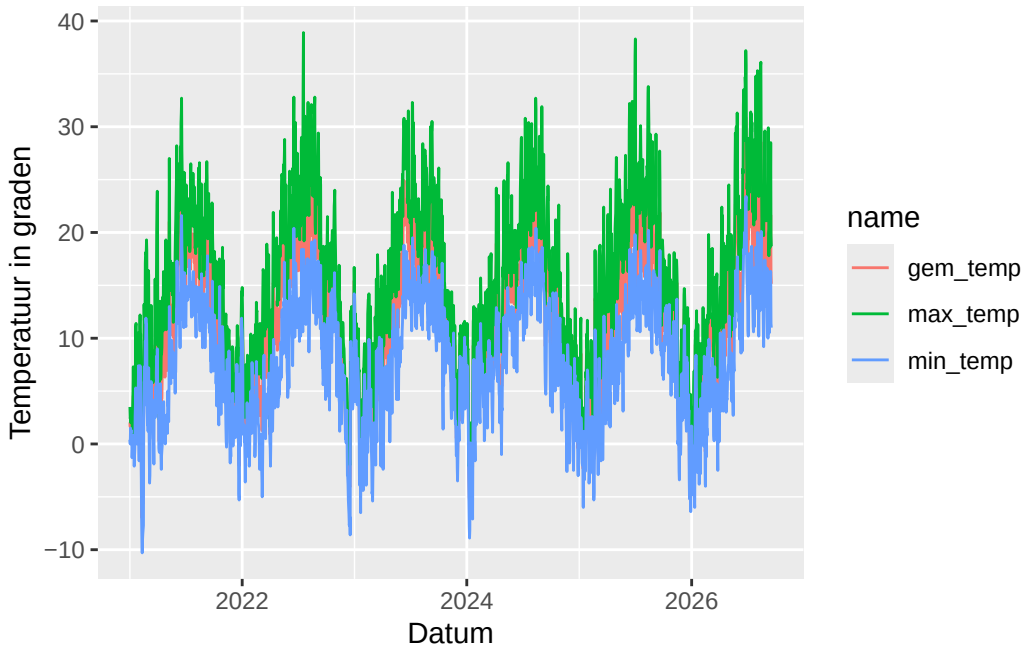
```
maastricht_2021_2030 |>
  group_by(datum) |>
  summarise(gem_temp = mean(temperatuur_graden, na.rm = TRUE),
            max_temp = max(temperatuur_graden, na.rm = TRUE),
            min_temp = min(temperatuur_graden, na.rm = TRUE)) |>
  select(datum, gem_temp, min_temp, max_temp) |>
  pivot_longer(cols = c(gem_temp, max_temp, min_temp))
```

```
# A tibble: 6,258 x 3
   datum      name     value
   <date>     <chr>    <dbl>
```

```
 1 2021-01-01 gem_temp  2.04
 2 2021-01-01 max_temp  3.5
 3 2021-01-01 min_temp  0.4
 4 2021-01-02 gem_temp  1.62
 5 2021-01-02 max_temp  2.4
 6 2021-01-02 min_temp  0
 7 2021-01-03 gem_temp  2.14
 8 2021-01-03 max_temp  2.6
 9 2021-01-03 min_temp  1.5
10 2021-01-04 gem_temp  1.52
# i 6,248 more rows
```

Dit is dan weer goed om te zetten naar een grafiek en de kleur van de lijn zal bepaald worden voor de waarde in de kolom `name`.

```
maastricht_2021_2030 |>
  group_by(datum) |>
  summarise(gem_temp = mean(temperatuur_graden, na.rm = TRUE),
            max_temp = max(temperatuur_graden, na.rm = TRUE),
            min_temp = min(temperatuur_graden, na.rm = TRUE)) |>
  select(datum, gem_temp, min_temp, max_temp) |>
  pivot_longer(cols = c(gem_temp, max_temp, min_temp)) |>
  ggplot(aes(x=datum, y = value)) + geom_line(aes(colour = name)) +
    labs(x="Datum", y="Temperatuur in graden")
```

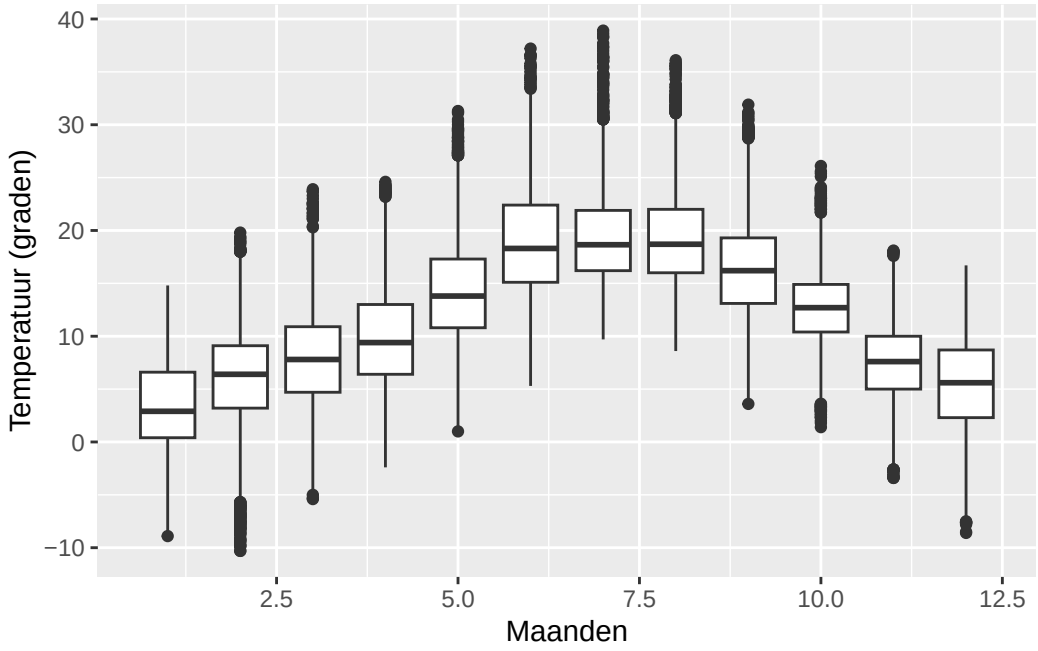


3.4.1 Nu even zelf nadenken

1. Maak de bovenstaande grafiek voor het jaar 2021.
2. Maak dezelfde grafiek maar dan voor het jaar 2025.
3. Maak dezelfde grafiek maar dan voor het huidige jaar (is weliswaar nog niet afgelopen).
4. Wat valt op wanneer je de verschillende jaren vergelijkt?

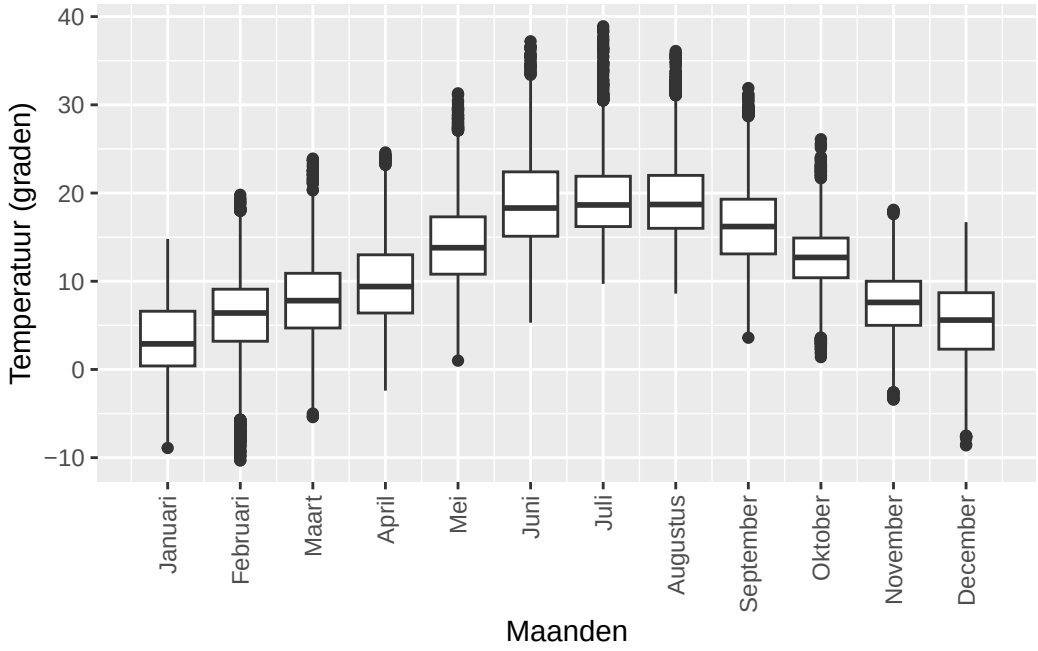
We hebben een aantal trends gezien van verschillende jaren. Je zou dit ook kunnen laten zien door de data te groeperen per maand en de daggemiddeldes in een boxplot te zetten:

```
maastricht_2021_2030 |>
  group_by(month(datum)) |>
  ggplot(aes(x=month(datum), y = temperatuur_graden)) + geom_boxplot(aes(group = month(datum)
  labs(x="Maanden", y = "Temperatuur (graden)")
```



Hier zijn echter de zogenaamde ticks op de x-as niet heel mooi, maar gelukkig kun je dat eenvoudig aanpassen. Dat is wel eenmalig wat typewerk. We gebruiken `scale_x_continuous` omdat de maanden worden omgezet naar een continue schaal in de software. Door de groepering maken het de eenheden echter discreet. Voor de leesbaarheid worden de tekstlabels nog even gedraaid met `+ theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))`.

```
maastricht_2021_2030 |>
  group_by(month(datum)) |>
  ggplot(aes(x=month(datum), y = temperatuur_graden)) + geom_boxplot(aes(group = month(datum)
  labs(x="Maanden", y = "Temperatuur (graden)") +
  scale_x_continuous(
    breaks = 1:12,
    labels = c("Januari", "Februari", "Maart", "April", "Mei","Juni", "Juli", "Augustus", "Se
  ) +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```



Door deze grafiek te maken worden de meerjarige trends in de temperatuur duidelijk en begin je een idee te krijgen van het klimaat in onze regio, waarbij de zomers gemiddeld warm zijn en de winters kouder.

3.4.2 Zelf aan de slag

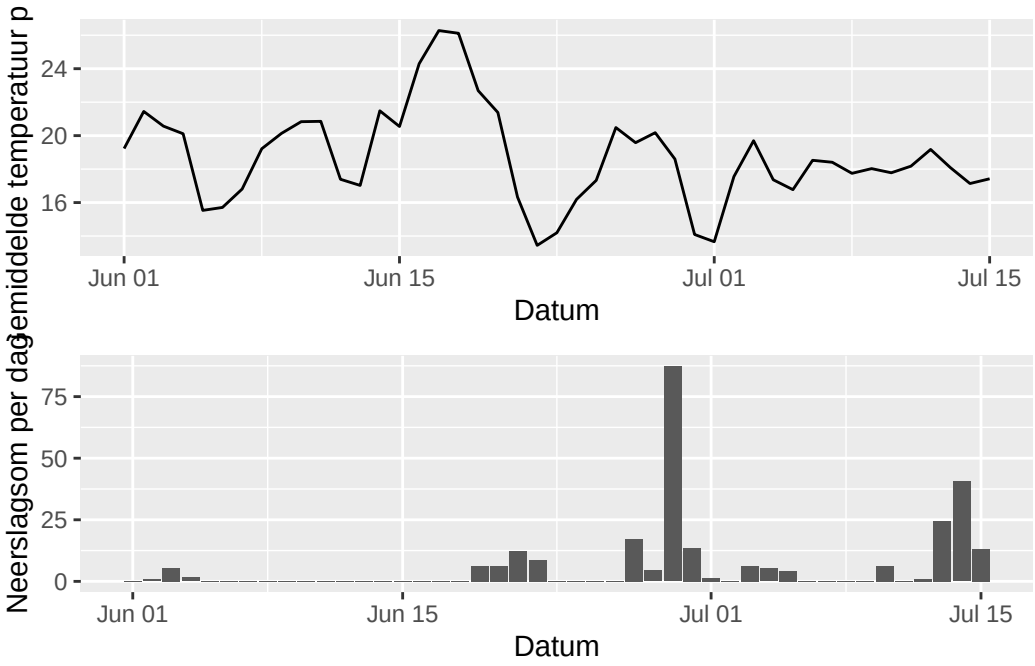
1. Op welke datum is de hoogste temperatuur gemeten in deze periode?
2. Op welke datum is de laagste temperatuur gemeten?
3. In welke jaren vielen de 10 heetste dagen?
4. Maak een grafiek met daarin alleen temperaturen die meer dan 2 standaarddeviaties boven het gemiddelde liggen.

3.5 Grafieken combineren

We hebben hiervoor een grafiek van de neerslag gemaakt en gezien dat in een bepaalde periode in 2021 een extreme hoeveelheid neerslag is geweest. We kunnen nu ook een grafiek maken waarbij en de neerslag en de temperatuur op die dagen te zien is. Het `ggpubr` package biedt mogelijkheden om grafieken te combineren. Hiertoe gebruiken we `ggarrange()`, waarbij de syntax is `ggarrange(grafiek_1, grafiek_2, ncol = 1)`. Op deze manier gebruikt hij twee grafieken en zet ze onder elkaar. Je kunt ze met `nrow = 1` ook naast elkaar zetten. Met deze

functie kun je vele grafieken tegelijkertijd laten zien, waarbij de assen op elkaar afgestemd zijn.

```
ggarrange(
  maastricht_2021_2030 |>
    group_by(datum) |>
    filter(YYYYMMDD >= 20210600 & YYYYMMDD <= 20210715) |>
    summarise(gem_temp = mean(temperatuur_graden, na.rm = TRUE)) |>
    ggplot(aes(x=datum, y = gem_temp)) + geom_line() +
      labs(x="Datum", y = "Gemiddelde temperatuur per dag")
  ,
  maastricht_2021_2030 |>
    group_by(datum) |>
    filter(YYYYMMDD >= 20210600 & YYYYMMDD <= 20210715) |>
    summarise(neerslagsom = sum(neerslag_mm, na.rm = TRUE)) |>
    ggplot(aes(x=datum, y = neerslagsom)) + geom_bar(stat = "Identity") +
      labs(x="Datum", y = "Neerslagsom per dag")
  , ncol = 1
)
```



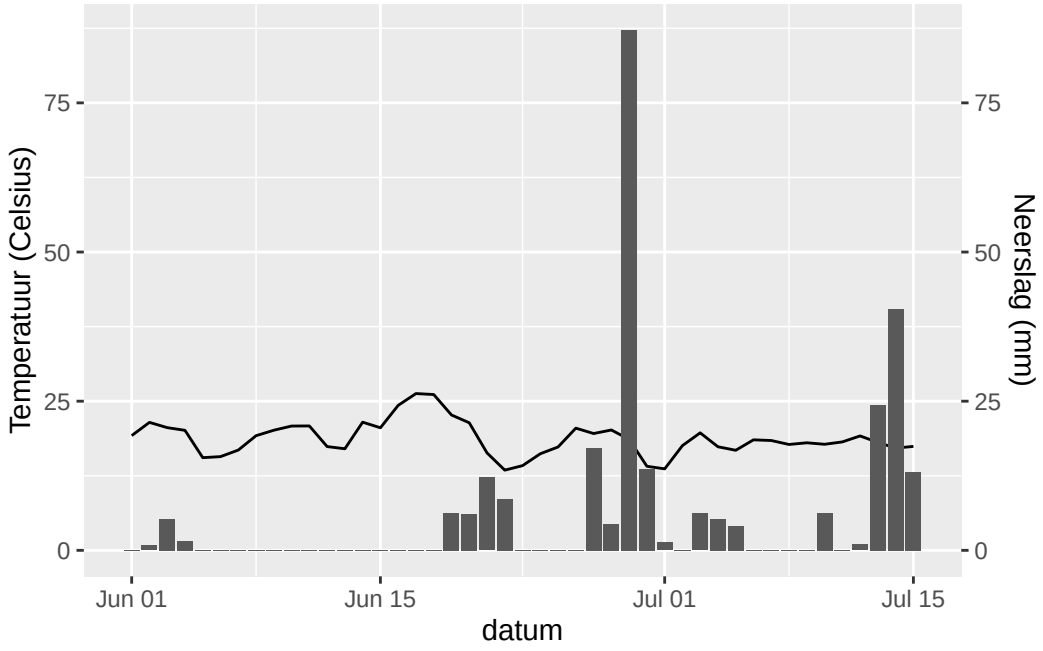
3.5.1 Zelf aan de slag

1. Maak de bovenstaande grafiek voor januari 2026. Wat valt op?

2. Maak dezelfde grafiek, maar zet nu de neerslag boven en de temperatuur boven.
3. Maak dezelfde grafiek, maar zorg dat de grafieken naast elkaar staan.

Het is ook mogelijk om twee verschillende grafieken te combineren in één grafiek, door bijvoorbeeld eerst een `geom_line()` te maken en daarna met andere data een `geom_bar()` hieraan toe te voegen. Op deze manier krijg je alle data samen in één grafiek. In het onderstaande voorbeeld voegen we een extra y-as toe, ter verduidelijking, met `scale_y_continuous(sec.axis = sec_axis(~.,name = "Neerslag (mm)"))`.

```
maastricht_2021_2030 |>
  group_by(datum) |>
  filter(YYYYMMDD >= 20210600 & YYYYMMDD <= 20210715) |>
  summarise(gem_temp = mean(temperatuur_graden, na.rm = TRUE), neerslagsom = sum(neerslag_mm
  ggplot(aes(x=datum, y = gem_temp)) + geom_line() +
    geom_bar(aes(x=datum, y = neerslagsom), stat = "Identity") +
    scale_y_continuous(sec.axis = sec_axis(~.,name = "Neerslag (mm)")) +
    ylab("Temperatuur (Celsius)")
```



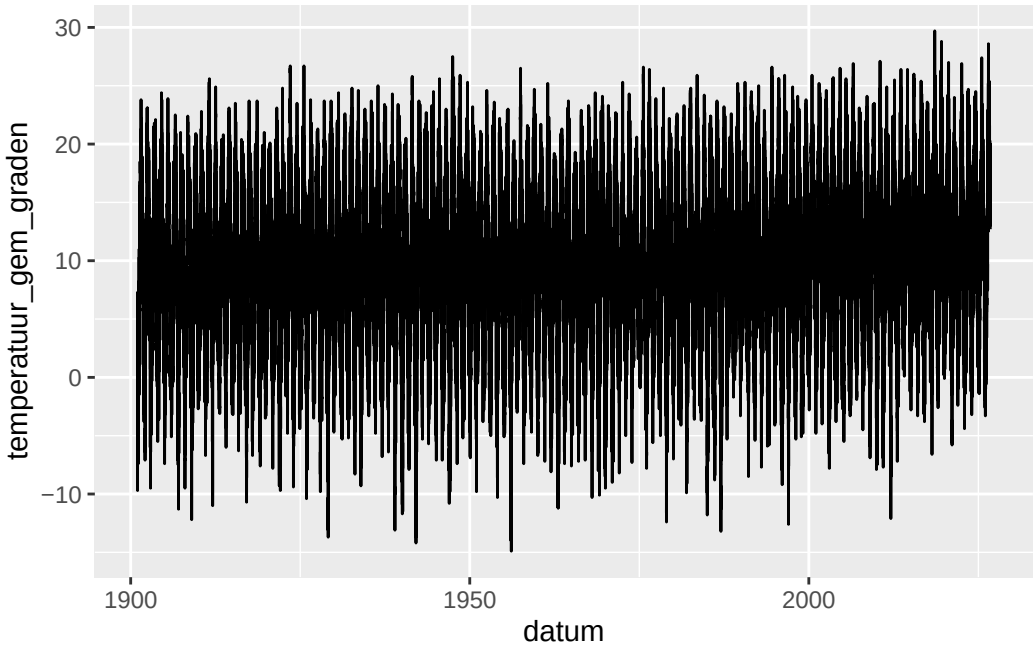
Welke grafiek vind jij duidelijker? Twee losse grafieken of een gecombineerde grafiek met verschillende y-assen? En waarom?

3.6 Klimaat: meerjarige data

We hebben nu vooral data van een enkele weerstation bekeken en de uurgegevens gecombineerd tot daggevens. Het KNMI biedt echter ook veel data aan over hele lange periodes. Het langst-functionerende weerstation is het station in De Bilt. Ga naar https://www.knmi.nl/nederland-nu/klimatologie/daggegevens en download daar de data van De Bilt. Het bestand heet `etmgeg_260.zip` pak het zip-bestand uit en plaats `etmgeg_260.txt` in de map met data. Lees dit bestand in en denk daarbij aan de voorgaande opdracht met betrekking tot de datum, de neerslag in mm en de temperatuur in graden. Zorg dus dat je een veld voor `datum` aanmaakt bij inlezen, een voor de neerslag in mm `neerslag_mm` en voor de gemiddelde dagtemperatuur (`temperatuur_gem_graden`). Let goed op hoeveel regels je moet overslaan

Hopelijk is dit geheel zelfstandig gelukt? Laten we eens een grafiek maken van de temperatuur van 1900 tot nu.

```
de_bilt |>
  ggplot(aes(x=datum, y=temperatuur_gem_graden)) + geom_line()
```



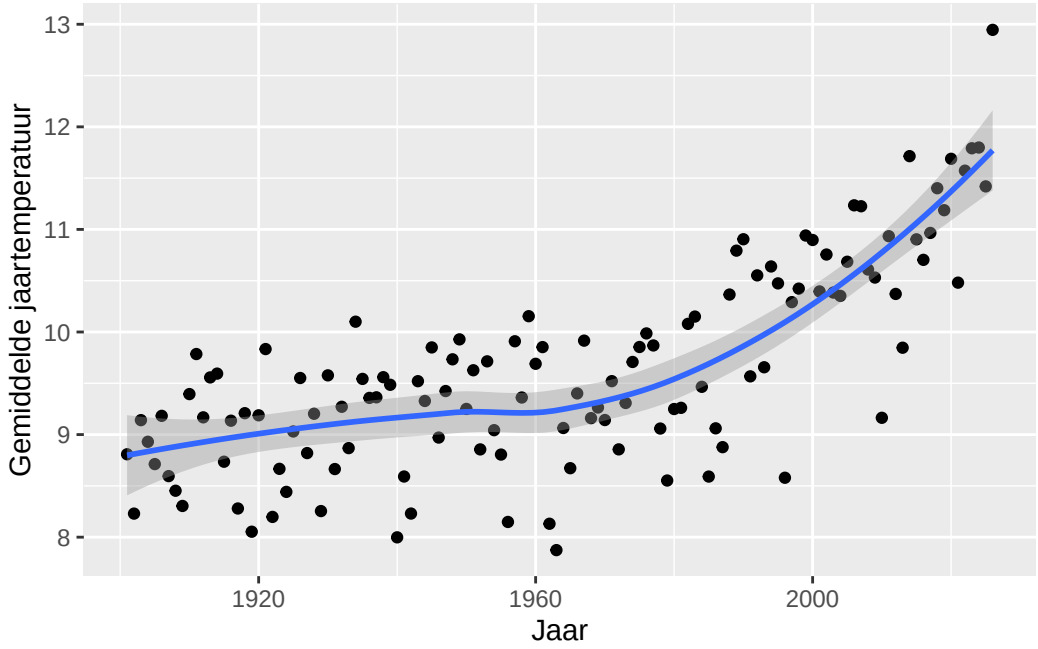
3.6.1 Zelf doen:

1. Dit is nog niet heel duidelijk dus voeg zelf even een trendlijn toe, om de trend te zien.

2. Dit is nog steeds niet zo mooi, dus verander de grafiek van een lijndiagram naar een spreidingsdiagram met punten.

Deze trendlijn laat goed zien dat er een opwaartse trend in de gemiddelde dagtemperatuur zit, maar echt een sterkte trend zien we niet. Hierbij speelt ook dat de temperatuur in de loop van het jaar in de verschillende seizoenen verandert. Daarom gaan we nu de data groeperen op jaar en dan de gemiddelde temperatuur per jaar berekenen en visualiseren in een grafiek. Dat doen we aldus:

```
de_bilt |>
  group_by(year(datum)) |>
  summarise(gemiddeld_jaar = mean(temperatuur_gem_graden)) |>
  setNames(c("jaar", "gemiddelde_jaartemperatuur")) |>
  ggplot(aes(x=jaar, y=gemiddelde_jaartemperatuur)) + geom_point(aes()) +
    geom_smooth() + labs(x="Jaar", y="Gemiddelde jaartemperatuur")
```



Let hierbij op, want door te groeperen en daarna een gemiddelde te berekenen heeft de kolom met het jaar een nieuwe interne naam gekregen die onhandig is. Daarom helpen we onszelf door de beide kolommen een nieuwe naam te geven met de functie `setNames(c("jaar", "gemiddelde_jaartemperatuur"))`.

De nu gemaakte grafiek laat een veel sterkere verandering van het klimaat zien en dat de gemiddelde jaartemperatuur sinds de jaren '60 zeer sterk is gestegen, iets wat we kennen als klimaatverandering.

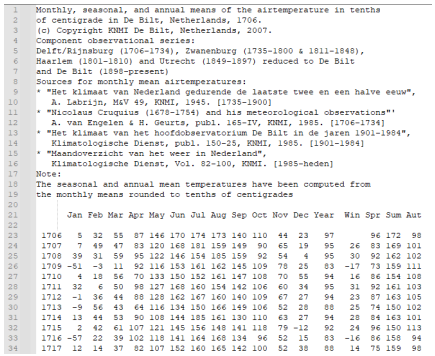
3.6.2 En dan weer zelf aan de slag

1. Maak een grafiek van de gemiddelde jaarlijkse neerslag. Zijn daar trends in te zien?
2. Maak een visualisatie van zowel de grafiek met de gemiddelde jaarlijkse temperatuur en die van de neerslag, waarbij deze grafieken boven elkaar staan.
3. Er zijn bepaalde trends zichtbaar. Wat betekent dat voor jouw onderzoeksgebied? Welke aanpassingen zijn nodig in het onderzoeksgebied?

3.7 Historische klimaatdata toevoegen

De metingen in De Bilt worden sinds 1901 goed bijgehouden, maar er bestaan ook oudere metingen, welke ons een goed beeld kunnen geven van de veranderingen van het klimaat over de afgelopen eeuwen. Deze biedt het KNMI ook kostenloos aan via: https://www.knmi.nl/nederland-nu/klimatologie/daggegevens/antieke-waarnemingen. Ze bieden diverse databronnen aan en we gebruiken nu de langstlopende uit Utrecht: https://cdn.knmi.nl/knmi/map/page/klimatologie/gegevens/daggegevens/antieke_wrn/labrijn_ea.zip. Pak deze zip uit en plaats het bestand `labrijn_ea.dat` in de `data`-map.

Het bestand `labrijn_ea.dat` lijkt een wat exotisch formaat, maar als je deze in een tekst-editor bekijkt, lijkt het gewoon een soort CSV-bestand, maar dan met een andere extensie.



```
Monthly, seasonal, and annual means of the airtemperature in tenths
of centigrade in De Bilt, Netherlands, 1706.
(c) Copyright KNMI De Bilt, Netherlands, 2007.
Component observational series:
Delft/Rijnsburg (1706-1734), Zwanenburg (1735-1800 & 1811-1848),
Haarlem (1801-1810) and Utrecht (1849-1897) reduced to De Bilt
and De Bilt (1898-present)
Sources for monthly mean airtemperatures:
* "Het klimaat van Nederland gedurende de laatste twee en een halve eeuw",
   A. Labrijn, MnV 49, KNMI, 1945. [1735-1900]
* "Nicolaus Cruquius (1678-1754) and his meteorological observations"
   A. van Engelen & H. Geurts, publ. 165-IV, KNMI, 1985. [1706-1734]
* "Het klimaat van het hoofdobservatorium De Bilt in de jaren 1901-1984",
   Klimatologische Dienst, publ. 150-25, KNMI, 1985. [1901-1984]
* "Maandoverzicht van het weer in Nederland",
   Klimatologische Dienst, Vol. 82-100, KNMI. [1985-heden]
Note:
The seasonal and annual mean temperatures have been computed from
the monthly means rounded to tenths of centigrades

      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec Year  Win Spr Sum Aut
1706   5  32  55  87 146 170 174 173 140 110  44  23   97       96 172  98
1707   7  49  47  83 120 168 181 159 149  90  65  19   95   26  83 169 101
1708  39  31  59  95 122 146 154 185 159  92  54   4   95   30  92 162 102
1709 -51  -3  11  92 116 153 161 162 145 109  78  25   83  -17  73 159 111
1710   4  18  56  70 133 150 152 161 147 108  70  55   94   16  86 154 108
1711  32   6  50  98 127 168 160 154 142 106  60  34   95   31  92 161 103
1712  -1  36  44  88 128 162 167 160 140 109  67  27   94   23  87 163 105
1713  -9  56  43  64 116 134 150 166 149 106  52  28   88   25  74 150 102
1714  13  44  53  90 108 144 185 161 130 110  63  27   94   28  84 163 101
1715   2  42  61 107 121 145 156 148 141 118  79 -12   92   24  96 150 113
1716 -57  22  39 102 118 141 164 168 134  96  52  15   83  -16  86 158  94
1717  12  14  37  82 107 152 160 165 142 100  52  38   88   14  75 159  98
```

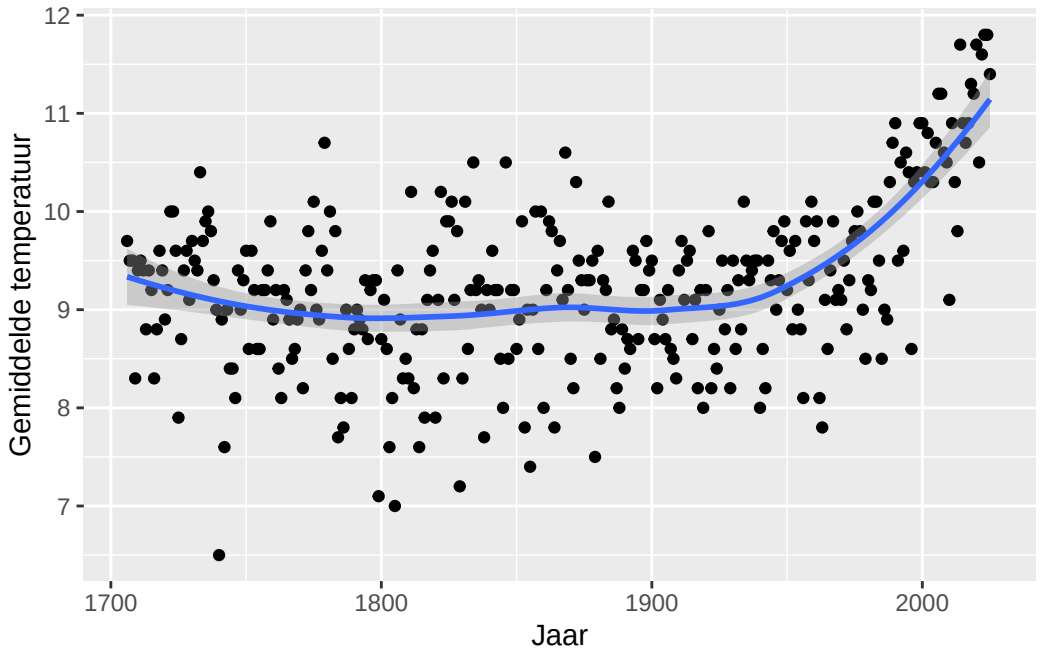
Figure 3.2: De data in het bestand `labrijn_ea.dat`

Het is opvallen dat de data niet door een symbool wordt gescheiden, maar dat er gebruik gemaakt wordt van vaste kolombreedtes. Dat is niet goed in te lezen met `read.csv()`, maar gelukkig zijn er andere tools. De readr-library (Wickham et al. 2026) heeft de functie `read_fwf()` die gemaakt is voor het inlezen van fixed-width-files. De titels van de velden staan weer wat onhandig met gewoon inlezen, dat de die eerst skippen en daarna opnieuw aanmaken met `setNames()`

```
labrijn <- read_fwf("data/labrijn_ea.dat", skip = 22) |>
  setNames(c("Jaar", "Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "
```

Met deze data kunnen we nu eenvoudig verschillende grafieken maken. Laten we beginnen met de gemiddelde jaartemperatuur, met een langjarige trendlijn.

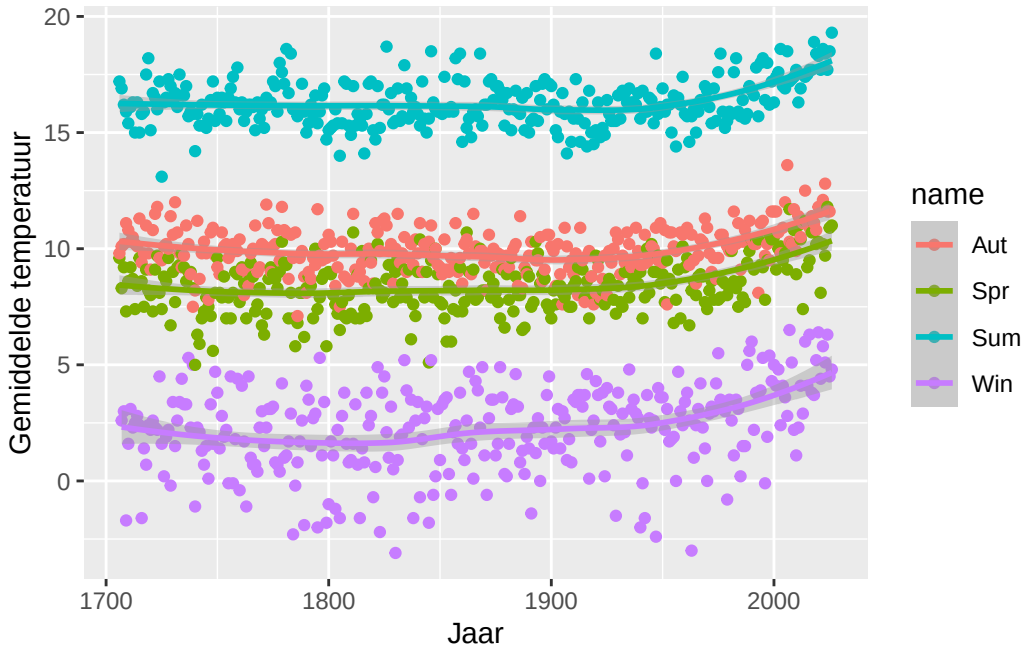
```
labrijn |>
  ggplot(aes(x=Jaar, y=Year_temp/10)) + geom_point() + geom_smooth() +
    labs(y="Gemiddelde temperatuur")
```



Nu wordt nog duidelijker welke enorme gemiddelde opwarming van de aarde op dit moment gaande is en dat die met name sinds het midden van de twintigste eeuw uitermate hard stijgt. We zien ook een grote variatie van jaar tot jaar, maar dat de afgelopen jaren de variatie eigenlijk steeds minder aan het worden is

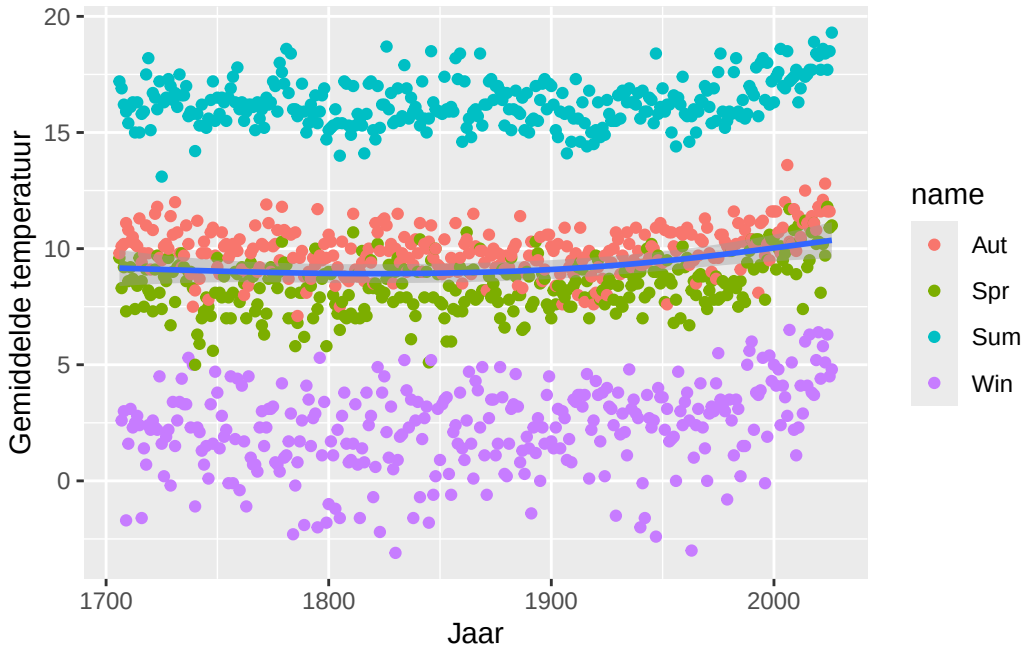
Het is ook mogelijk om eens te kijken of de temperaturen in de verschillende seizoenen in deze dataset anders toenemen. Daartoe moeten we weer gebruik maken van de `pivot_longer()` optie. Daarnaast zijn alle temperaturen weer in 0.1 graden Celsius, dus ook hier weer omrekenen. Door de kleur te bepalen bij de *aesthetics* bij het aanroepen van ggplot, worden de trendlijnen automatisch per seizoen bepaald.

```
labrijn |>
  pivot_longer(cols = c("Win", "Spr", "Sum", "Aut")) |>
  ggplot(aes(x=Jaar, y=value/10, colour=name)) + geom_point() + geom_smooth() +
  labs(y="Gemiddelde temperatuur")
```



Als je de de kleur alleen voor de punten instelt (bij `geom_point()`), zal de algemene trendlijn van alle seizoenen worden bepaald. Dat is in dit geval net geheel logisch, maar kan wel.

```
labrijn |>
  pivot_longer(cols = c("Win", "Spr", "Sum", "Aut")) |>
  ggplot(aes(x=Jaar, y=value/10)) + geom_point(aes(colour=name)) + geom_smooth() +
  labs(y="Gemiddelde temperatuur")
```



Deze laatste trendlijnen in de grafiek is dus minder goed en vertelt ook minder duidelijk een verhaal.

3.7.1 En nu zelf weer aan de slag.

1. Pas legenda van de grafiek met de trendlijnen per seizoen aan en zorg dat deze een duidelijke titel krijgt en dat de namen van de seizoenen uitgeschreven worden. Tip: maak gebruik van `scale_color_discrete()`.
2. Maak dezelfde grafiek maar dan voor de verschillende maanden van het jaar. Welke maanden laten de grootste veranderingen zien?
3. Wat was eigenlijk de warmste maand van de afgelopen eeuwen?
4. Wanneer waren de warmste 10 maanden?
5. Wanneer waren de koudste 10 maanden?

   Extra

6. Het WMO publiceert een hele bekende grafiek met de zogenaamde *Warming stripes* (https://wmo.int/site/science-action/warming-stripes). Dit laat de temperatuurverschillende zien ten opzichte van de gemiddelde temperatuur tussen 1850 en 1900 (https://wmo.int/media/news/wmo-publishes-global-update-of-climate-datasets). Maak een lijndiagram voor de periode 1850 tot nu die hetzelfde laat zien op basis van de `labrijn` dataset.

7. Lukt het ook om de *warming stripes* voor Nederland te maken?

3.8 Project

Denk voor het eigen project na over de volgende vragen.

- Welke invloed heeft het weer op de problemen in mijn onderzoeksgebied?
- Welke invloed heeft het klimaat op de problemen in mijn onderzoeksgebied?
- Welke klimaatsdata heb ik nodig voor mijn onderzoek?
- Er zijn diverse klimaatscenario's (https://www.knmi.nl/kennis-en-datacentrum/achtergrond/knmi-23-klimaatscenario-s). Welke invloeden kan je voor de toekomst onderscheiden die invloed hebben op het onderzoeksgebied?

Ga vast aan de slag om de relevante data te verzamelen en maak de benodigde analyses en grafieken.

4 Werkcollege 3: werken met ruimtelijke data

4.1 Inleiding

In de voorgaande werkcolleges hebben jullie gewerkt met grondwaterstanden uit verschillende bronnen en met gegevens over weer en klimaat. Deze week gaan we aan de slag met ruimtelijke data van PDOK en uit de BRO. We gaan er hierbij vanuit dat jullie de stof van de voorgaande weken hebben verwerkt. Daardoor zullen niet alle stappen worden uitgelegd. Na dit werkcollege heb je voldoende kennis en ervaring om zelf analyses uit te voeren op eigen datasets.

4.2 Project aanmaken

Maak een nieuw project aan voor dit werkcollege. Als je niet meer weet hoe dat moet, kijk dan even bij het eerste werkcollege.

Maak ook een nieuw script aan om in te werken en sla deze op in de projectmap. Zorg ook voor de standaardmappen `data` en `export`.

4.3 Bibliotheken

We laden weer een aantal benodigde bibliotheken. Daat zitten enkele nieuwe bij, die je dus moet installeren. We gebruiken in dit werkcollege:

- pdokr (Eisma 2026): voor het vinden en inlezen van data vanuit PDOK, inclusief BRO
- dplyr (Wickham, François, et al. 2023): voor het bewerken van tabulaire data (`tibble`)
- tmap (Tennekes 2018): voor het maken van kaarten
- sf (Pebesma 2018; Pebesma et al., n.d.); voor het werken met ruimtelijke data
- ggplot (Wickham 2016): voor het maken van grafieken

Als je grafieken moet maken, zorg dan dat je de juiste bibliotheken laadt! Je hebt er in de voorgaande lessen met een aantal leren werken

```r
# load libraries
library(pdokr)
library(dplyr)
library(tmap)
library(sf)
library(ggplot2)

# force non scientific formatting of numbers
options(scipen = 999)
options(digits=3)
```

4.4 Een eerste kaart maken

Voor dit werkcollege maken we gebruik van de `pdokr` package van Coen Eisma en naar voorbeeld van zijn blog (https://www.coeneisma.nl/blog/pdokr-op-cran/) maken we een kaart.

Op zijn blog schrijft hij als uitleg:

> PDOK (Publieke Dienstverlening Op de Kaart) is hét platform voor open geodata van de Nederlandse overheid: denk aan gemeente-, wijk- en buurtgrenzen van het CBS, gebouwen uit de BAG, topografie, luchtfoto's en nog veel meer. Allemaal gratis en zonder API-key. Toch blijft deze waardevolle databron voor veel data-analisten en -scientists vaak onaangeboord: wie niet dagelijks met geodata werkt, haakt af op termen als OGC-services, WFS-URL's en coördinaatstelsels. Dat is precies wat pdokr grotendeels wegneemt. Je zoekt een dataset (`pdok_search_datasets()`, `pdok_list_layers()`), leest een laag in met `pdok_read()`, en filtert ondertussen makkelijk op specifiek gebied of jaar. En adressen omzetten naar coördinaten, gemeente en terug kan met `pdok_geocode()` en `pdok_reverse_geocode()`. Wanneer je met pdokr en dataset inlaadt krijg je een zogenoemd sf-object: gewoon een dataframe met een geometriekolom en een sf-class. Je kunt er dus gewoon mee werken zoals je gewend bent — zoals filteren, joinen en samenvatten met dplyr — én er ruimtelijke analyses op doen met de functies uit de sf-package zoals oppervlaktes of afstanden berekenen. Of je eigen data koppelen aan officiële geodata." (Bron: https://www.coeneisma.nl/blog/pdokr-op-cran/)

We gaan nu een eenvoudige kaart maken van Apeldoorn. Daarom moeten we eerst weten waar deze stad ligt en gebruiken daarvoor `pdok_geocode()`

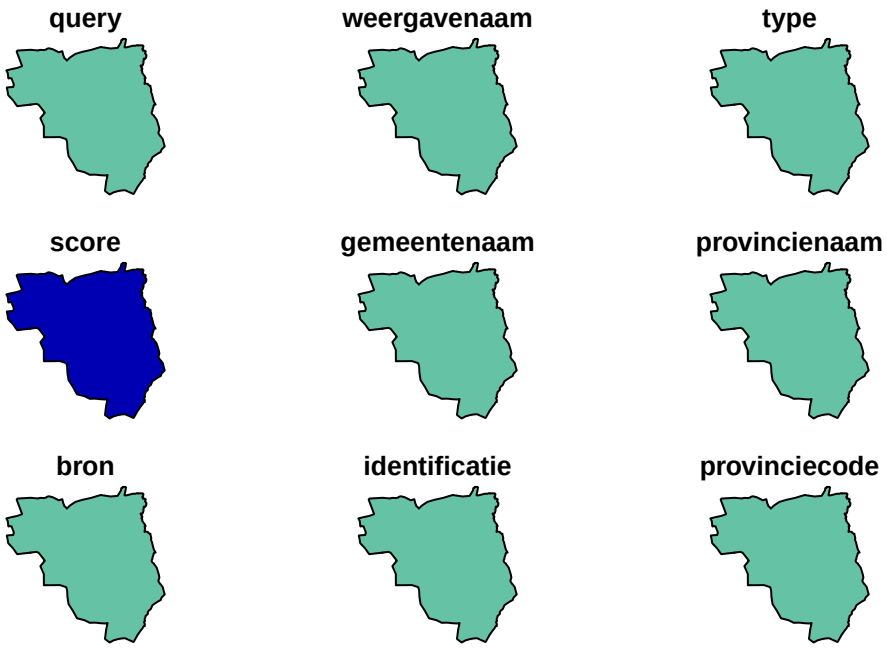
```r
Apeldoorn_grens <- pdok_geocode("Apeldoorn", "gemeente")
```

Op deze manier hebben we een simple-features (sf) object gemaakt. Dit is een ruimtelijke object met de grenzen van de gemeente Apeldoorn.

We zouden dit eenvoudig kunnen plotten met de standaard `plot()` functie.

```
plot(Apeldoorn_grens)
```

```
Warning: plotting the first 9 out of 16 attributes; use max.plot = 16 to plot
all
```



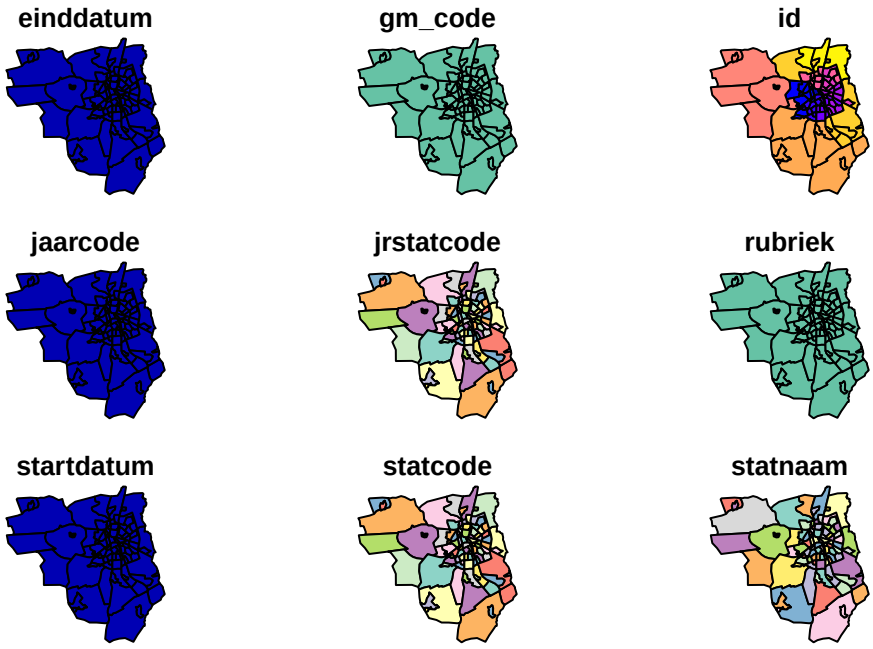
Deze plot dan voor de eerste 10 kolommen de data en de geografische grenzen en dat is in dit geval weinig nuttig, omdat het slechts 1 polygoon is.

Daarom gaan we eerst wat meer data opvragen. We vragen nu de data op van het CBS met informatie over de buurten van Apeldoorn. Met de onderstaande code gebruiken we de functie `pdok_read()` om deze data op te vragen, waarbij we - De laag "buurt_gegeneraliseerd" opvragen uit "cbs/gebiedsindelingen", - Deze data tot het jaar 2025 bepertken - Binnen de ruimtelijke gemeentegrenzen van Apeldoorn - En zorgen dat de gemeentecode gelijk is uit de laag met de grens van de gemeente

```
buurten <- pdok_read(
  "cbs/gebiedsindelingen", "buurt_gegeneraliseerd",
  datetime = 2025, filter_by = Apeldoorn_grens) |>
  filter(gm_code == paste0("GM", Apeldoorn_grens$gemeentecode))
```

Wanneer we nu de kaartlaag met de buurten van Apeldoorn plotten, zie je wel verschillen voor de verschillende kolommen in de tabel en dat de inhoud dus soms verschillend is en soms hetzelfde

```
plot(buurten)
```



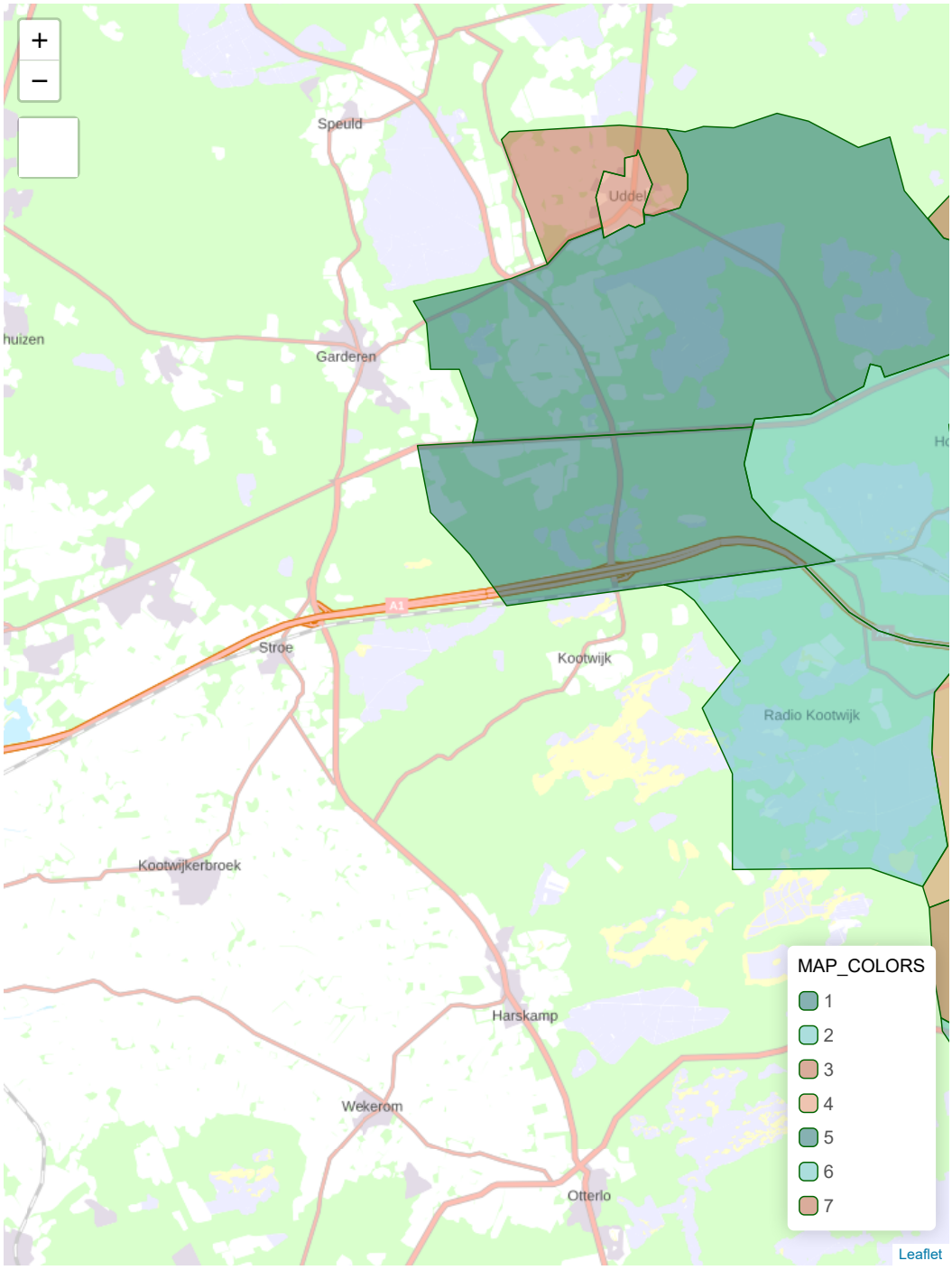
Het is nu met wat code vrij eenvoudig om een kaart te maken met we verschillende buurten in Apeldoorn door gebruik te maken van `tmap`. In het onderstaande voorbeeld kleuren we de buurten op basis van het veld `statnaam`.

```
tmap_mode("view")
```

```
i tmap modes "plot" - "view"
i toggle with `tmap::ttm()`
```

```
tm_basemap(pdok_basemap("pastel")) +
  tm_shape(buurten) +
   tm_polygons(
      fill = "MAP_COLORS",
      fill.scale = tm_scale_categorical(
        values = c("#0E6465", "#58BDBB", "#B65A38", "#E08A63")
      ),
```

```
    fill_alpha = 0.5, col = "darkgreen", id = "statnaam",
    fill.legend = tm_legend(show = TRUE)
  )
```


Speuld
Garderen
Stroe
Kootwijk
Radio Kootwijk
Kootwijkerbroek
Harskamp
Wekerom
Otterlo
A1
MAP_COLORS
1
2
3
4
5
6
7
Leaflet

Dit is niet per se nuttig, maar laat wel zien wat je met deze relatief eenvoudige code kunt doen. Het is wel belangrijk om de syntax vast te begrijpen van `tmap`

`tm_basemap(pdok_basemap("pastel"))` zegt dat je een pastel-achtegrondkaart gebruikt dan komt er een + `tm_shape(buurten)` geeft aan dat je `buurten` gebruikt als kaartlaag, wat wordt gevolgd door het bijbehorende `tm_polygons(...)` waarmee je aangeeft hoe je deze laag wilt visualseren (kleuren, vormen, etc).

Het CBS biedt echter nog veel meer data aan over de bevolking in de wijken, waarmee je inzicht krijgt in meer statistische data over de omgeving, zoals bevolkingssamenstelling, Deze laag laden we nu even

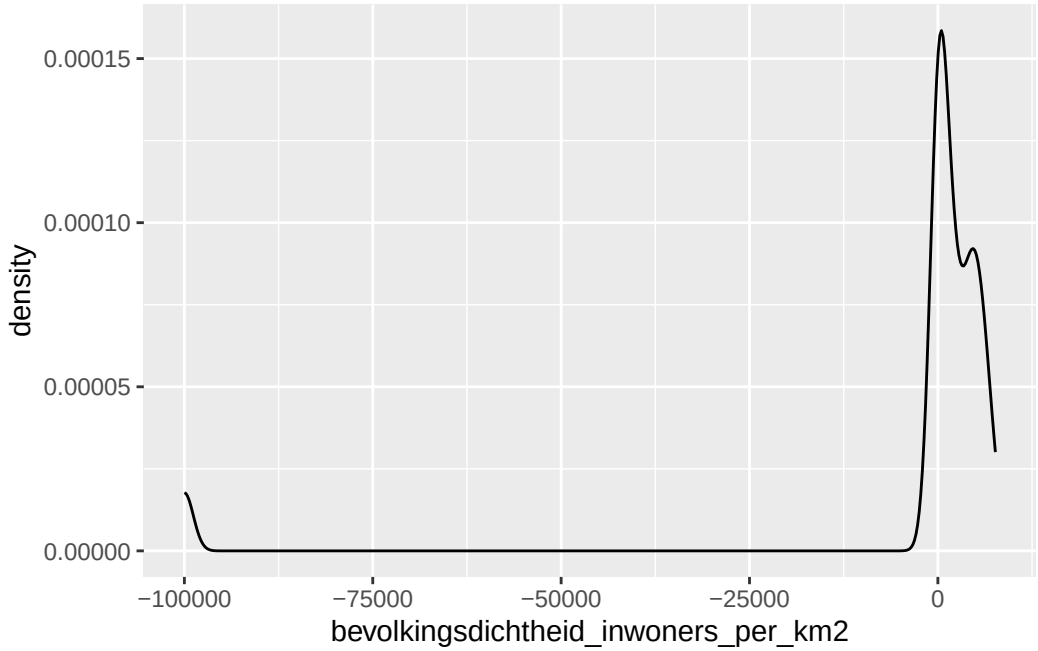
```
buurten2025 <- pdok_read(
  "cbs/wijken-en-buurten-2025", "buurten", filter_by = Apeldoorn_grens) |>
  filter(gemeentenaam == "Apeldoorn")
```

Het extra filter op gemeentenaam is nodig, omdat de ruimtelijke begrenzing met de gemeentegrens ook enkele polygonen van buurgemeentes selecteert.

Bekijk deze tabel eens even, om te zien welke data hierin zit.

Laten we eerst even een density-plot maken van de bevolkingsdichtheid. Dat is vergelijkbaar met een histogram.

```
buurten2025 |>
  ggplot(aes(x=bevolkingsdichtheid_inwoners_per_km2)) + geom_density()
```

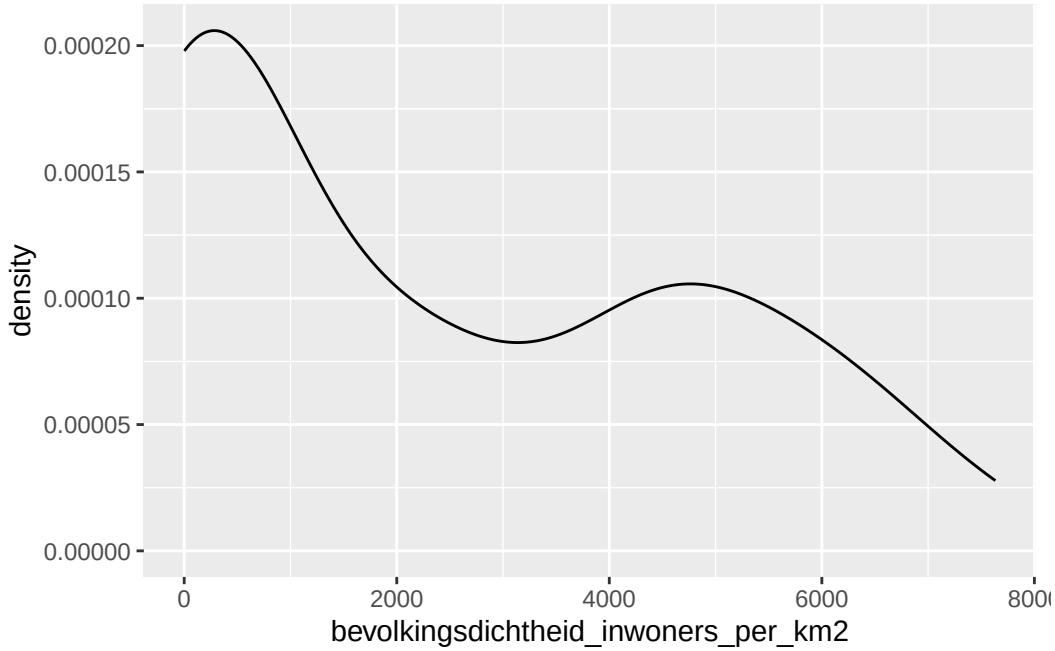


Hmm, deze data ziet er niet goed uit. Dat komt omdat de afwezigheid van data in deze dataset als een negatief getal wordt weergegeven. Dat is op te lossen door deze getallen met een `NA`te vervangen en daarna opnieuw de grafiek te maken. Op deze manier heb je de data opgeschoond en kun je duidelijke visualisaties maken:

```
buurten2025$bevolkingsdichtheid_inwoners_per_km2[buurten2025$bevolkingsdichtheid_inwoners_per

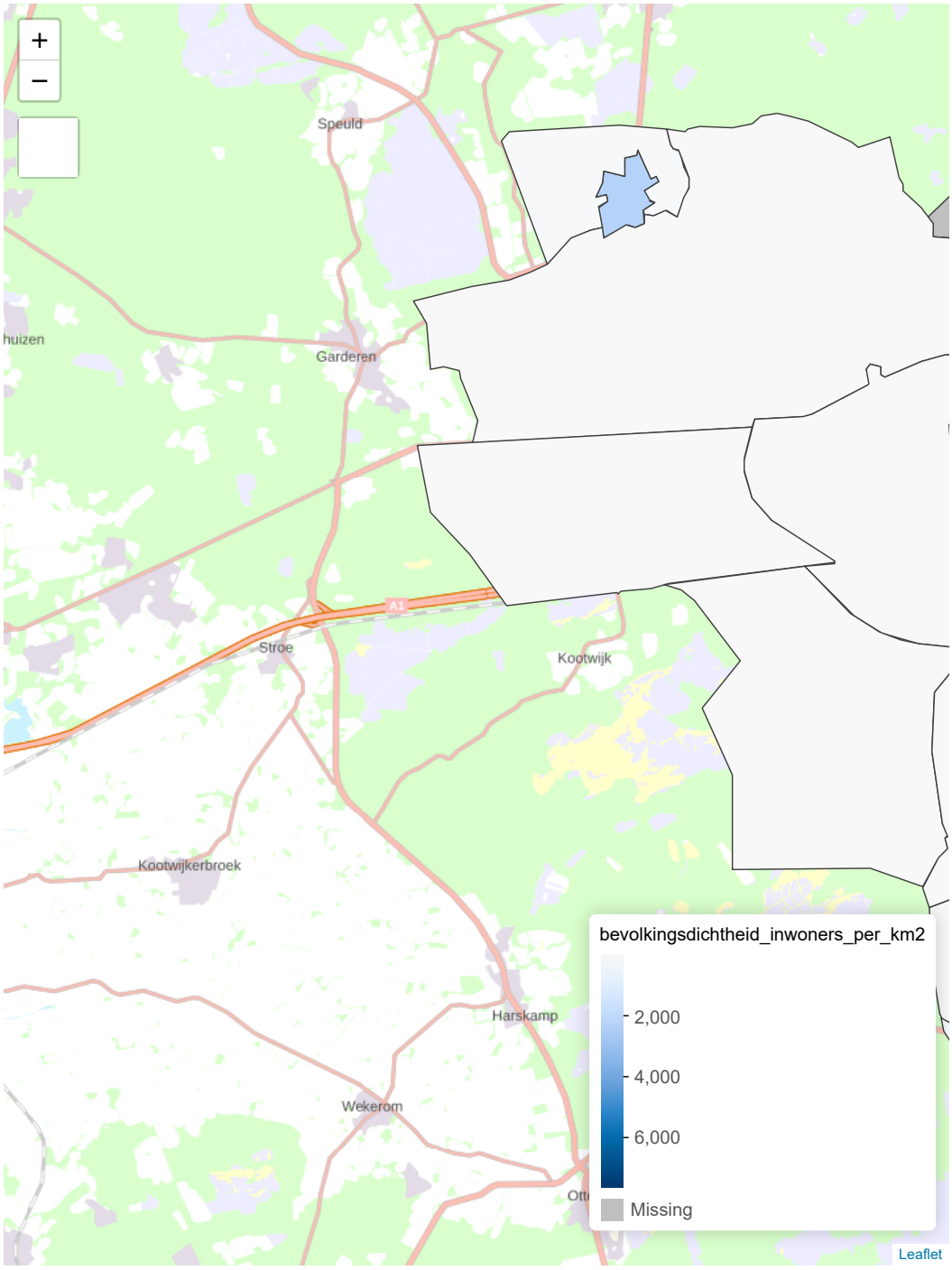
buurten2025 |>
  ggplot(aes(x=bevolkingsdichtheid_inwoners_per_km2)) + geom_density()
```

```
Warning: Removed 5 rows containing non-finite outside the scale range
(`stat_density()`).
```



We kunnen nu deze data ook in een kaart laten zien en dan zien we welke buurten dichter bevolkt zijn en welke minder. Deze gegevens helpen om een beeld te krijgen van een stad en ook te zien waar bepaalde ingrepen in de leefomgeving voor mensen verschillende invloeden kunnen hebben

```
tm_basemap(pdok_basemap("pastel")) +
  tm_shape(buurten2025) +
   tm_polygons(
      fill = "bevolkingsdichtheid_inwoners_per_km2",
      fill.scale = tm_scale_continuous(),
      fill.legend = tm_legend(show = TRUE)
    )
```


+
−
Speuld
huizen
Garderen
A1
Stroe
Kootwijk
Kootwijkerbroek
Harskamp
Wekerom
bevolkingsdichtheid_inwoners_per_km2
2,000
4,000
6,000
Missing
Leaflet

4.4.1 Nu zelf nadenken

Let op negatieve getallen bij het maken van de onderstaande opdrachten.

1. Maak een histogram voor deze bevolkingsdichtheid
2. In welke wijk wonen de meeste ouderen?
3. In welke wijk wonen de meeste kinderen?
4. Maak een grafiek van de bevolkingsopbouw van een wijk naar keuze.
5. Visualiseer in een kaart met de percentages van mensen tussen 25 en 45 jaar oud.
6. Visualiseer in een kaart de verhouding tussen mannen en vrouwen in de verschillende buurten laat zien.
7. Maak een staafdiagram waarin je voor twee verschillende buurten het aantal vrouwen en mannen met elkaar vergelijkt. Kies zelf de betreffende wijken.

4.5 Een kaart van Delft

Laten we eens een andere kaart maken, maar nu van de gemeente Delft. Daartoe moeten we eerst de gemeentegrenzen opvragen en daarna de grenzen van Delft bepalen.

```
gemeenten <- pdok_read(
  "cbs/gebiedsindelingen", "gemeente_gegeneraliseerd", datetime = 2025
)
delft <- gemeenten[gemeenten$statnaam == "Delft", ]
```

We gaan nu een kaart maken van het centrum van Delft en moeten dus weten welke wijken er in Delft zijn, om daarna het centrum te selecteren

```
wijken_delft <- pdok_read(
  "cbs/gebiedsindelingen", "wijk_gegeneraliseerd", datetime = 2025,
  filter_by = delft, predicate = "within"
)
binnenstad_delft <- wijken_delft[wijken_delft$statnaam == "Wijk 11 Binnenstad", ]
```

We hebben nu een ruimtelijk object met daarin de grenzen van de binnenstad van Delft en om een kaart te maken hebben we meer data nodig en daarom zoeken we de topografische data van alle gebouwen op en slaan dit op als een apart object. Dit betreft de topografische kaart (1:10.000)

```
gebouwen_delft <- pdok_read("kadaster/brt-top10nl", "gebouw_vlak", filter_by = binnenstad_del
```

We gaan nu een kaartje maken van de data die we verzameld hebben. Dat kan met de onderstaande code. - We gebruiken een grijze achtergrondkaart - De laag met de omtrek van de binnenstad voegen we daarna toe in de kleur grijs - De laag met gebouwen voegen we daarna toe, waarbij we de rand wit maken en de kleur (`col`) een soort oranje - We geven de kaart ook een titel - Tot slot nog een bronvermelding met betrekking tot de data. Dat is nu vrij kort, maar moet bij echt gebruik netter zijn, want het moet duidelijk zijn waar de data te vinden is. Dat kan ook vaak in het bijschrift. Voor meer informatie over de Topografische Kaart: https://www.pdok.nl/introductie/-/article/basisregistratie-topografie-brt-topnl

```
# Map it
tmap_mode("plot")
```

```
i tmap modes "plot" - "view"
```

```
tm_basemap(pdok_basemap("grijs")) +
  tm_shape(binnenstad_delft) +
    tm_borders(col = "grey40") +
  tm_shape(gebouwen_delft) +
    tm_polygons(fill = "#d95f02", col = "white", lwd = 0.4) +
  tm_title("Gebouwen in het historische centrum van Delft") +
  tm_credits("Kaartgegevens © Kadaster")
```

```
[basemaps] Tiles from "id_1",
"https://service.pdok.nl/brt/achtergrondkaart/wmts/v2_0/grijs/EPSG:3857/{z}/{x}/{y}.png",
"c(\"a\", \"b\", \"c\")", and "" at zoom level 15 couldn't be loaded
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
This message is displayed once per session.
```

Gebouwen in het historische centrum van Delft



4.6 Grondwaterpunten in Delft op de kaart

We hebben nu een aantal datasets gebruikt, maar gaan nu zelf op zoek naar data om daarmee te werken.

> **Datasets vinden**
>
> Je kunt zelf met de volgende functies zoeken naar datasets en/of layers:
>
> - `pdok_search_datasets("datasetnaam")` zoeken naar datasets in PDOK.
> - `pdok_search_layers("kadaster/brt-top10nl")` gebruik je om lagen in een dataset te zoeken. In het voorbeeld zoek je naar lagen in de dataset `kadaster/brt-top10nl`
>
> Uiteraard kun je de resultaten opslaan in een aparte data.frame of tibble, zodat je makkelijker kunt zoeken dan in de console.

We hebben in het vorige college gebruik gemaakt van grondwaterstanden. Die staan ook in de BRO en kun je ook via PDOK vinden. Met de onderstaande code vinden we datasets over grondwater

```
pdok_search_datasets("grondwater")
```

```
# A tibble: 1 x 7
  id                           name  description keywords services owner ogc_url
  <chr>                        <chr> <chr>       <list>   <chr>    <chr> <chr>
1 tno/bro-grondwatermonitorin~ BRO ~ Deze datas~ <chr>    ogc      tno   https:~
```

Dat levert echter heel veel data op. De kolom met `id` is belangrijk voor de naam. We kunnen meteen in de vraag deze kolom opvragen door de onderstaande code

```
pdok_search_datasets("grondwater")$id
```

```
[1] "tno/bro-grondwatermonitoring-in-samenhang-karakteristieken"
```

We weten nu dat het gaat om de dataset: "tno/bro-grondwatermonitoring-in-samenhang-karakteristieken". We kunnen met de functie 'pdok_list_layers()' een overzicht krijgen van alle lagen

```
pdok_list_layers("tno/bro-grondwatermonitoring-in-samenhang-karakteristieken")
```

```
# A tibble: 7 x 9
  dataset  layer title description start_date end_date crs    storage_crs bbox
  <chr>    <chr> <chr> <chr>       <date>     <date>   <list>       <int> <list>
1 tno/bro~ gm_g~ Gron~ De locatie~ NA         NA       <int>         4258 <dbl>
2 tno/bro~ gm_g~ GMW ~ De meest k~ NA         NA       <int>         4258 <dbl>
3 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>         4258 <dbl>
4 tno/bro~ gm_g~ GMN ~ De meest k~ NA         NA       <int>         4258 <dbl>
5 tno/bro~ gm_g~ GM -~ In de gege~ NA         NA       <int>         4258 <dbl>
6 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>         4258 <dbl>
7 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>           NA <dbl>
```

Dat lebvert een uitgebreide tabel op met veel informatie over de verschillende lagen

```
pdok_list_layers("tno/bro-grondwatermonitoring-in-samenhang-karakteristieken")
```

```
# A tibble: 7 x 9
  dataset  layer title description start_date end_date crs    storage_crs bbox
  <chr>    <chr> <chr> <chr>       <date>     <date>   <list>       <int> <list>
1 tno/bro~ gm_g~ Gron~ De locatie~ NA         NA       <int>         4258 <dbl>
```

```
2 tno/bro~ gm_g~ GMW ~ De meest k~ NA         NA       <int>         4258 <dbl>
3 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>         4258 <dbl>
4 tno/bro~ gm_g~ GMN ~ De meest k~ NA         NA       <int>         4258 <dbl>
5 tno/bro~ gm_g~ GM -~ In de gege~ NA         NA       <int>         4258 <dbl>
6 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>         4258 <dbl>
7 tno/bro~ gm_g~ Gron~ De meest k~ NA         NA       <int>           NA <dbl>
```

Een van de handige dingen van R is dat je ook functies kunt combineren en daarmee het aantal regels verminderen. Daarnaast schrijven we nu de resultaten weg in gw_lagen, wat ons de mogelijkheid geeft om meer informatie te lezen over de lagen:

```
gw_lagen <- pdok_list_layers(pdok_search_datasets("grondwater")$id)
```

De inhoud van deze tabel staat hieronder

dataset	layer	title	description	start_date	end_date	data_storage_bbox
tno/bro-grondwatermonitoring-in-samenhang-karakteristieken	gm_gmw_monitoringput	Grondwatermonitoringput (GMW)	De locatie en meest karakteristieke (ken)gegevens van nederlandse grondwatermonitoringputten (GMW) uit de Basisregistratie Ondergrond (BRO). Een grondwatermonitoringput is een werk dat in een geboord gat is gerealiseerd om gedurende langere tijd veranderingen in het grondwater in een specifiek deel van de ondergrond te kunnen registreren. Gewoonlijk bestaat een put uit 1 of meerdere monitoringsbuizen die van het oppervlak door een constructie worden beschermd. De GMW gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 4258, 28992, 3857, 4258	1.66, 48.04, 12.43, 56.11
tno/bro-grondwatermonitoring-in-samenhang-karakteristieken	gm_gmw_monitoringput_monitoringbuis	GMW Monitoringbuis	De meest karakteristieke (ken)gegevens van de monitoringbuizen van nederlandse grondwatermonitoringputten (GMW) uit de Basisregistratie Ondergrond (BRO). Bij een put met meerdere monitoringbuizen worden de gegevens per monitoringbuis weergegeven. De GMW gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 4258, 28992, 3857, 4258	1.66, 48.04, 12.43, 56.11

dataset	layer	title	description	start_date	end_date	storage_bbox
tno/brogm_grondwatermonitoringinsamenhangkarakteristieken	1	Grondwatermonitoringnetten (GMN)	De meest karakteristieke (ken)gegevens van nederlandse grondwatermonitoringnetten (GMN) uit de Basisregistratie Ondergrond (BRO). Een grondwatermonitoringnet is een verzameling meetpunten (gevormd door monitoringbuizen van grondwatermonitoringputten (GMW)), waarin onderzoek wordt gedaan om de toestand van het grondwater te kunnen bepalen en veranderingen erin te kunnen monitoren. De GMN gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 28992, 3857, 4258	4258, 1.66, 48.04, 12.43, 56.11
tno/brogm_grondwatermonitoringinsamenhangkarakteristieken	2	GMN_Meetpunt	De meest karakteristieke (ken)gegevens van afzonderlijke meetpunten van nederlandse grondwatermonitoringnetten (GMN) uit de Basisregistratie Ondergrond (BRO). Een meetpunt wordt gevormd door één monitoringbuis van een grondwatermonitoringput (GMW) waarin onderzoek wordt gedaan om de toestand van het grondwater te kunnen bepalen en veranderingen erin te kunnen monitoren. De GMN gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 28992, 3857, 4258	4258, 1.66, 48.04, 12.43, 56.11
tno/brogm_grondwatermonitoringinsamenhangwikarakteristieken	3	Verwijzing naar GMN	In de gegevensdefinitie van de registratieobjecten grondwaterstandonderzoek (GLD), grondwatersamenstellingsonderzoek (GAR) en formatieweerstand onderzoek (FRD) is een verwijzing opgenomen naar een grondwatermonitoringnet (GMN). In deze tabel/laag is deze verwijzing vastgelegd.	NA	NA	4326, 28992, 3857, 4258	4258, 1.66, 48.04, 12.43, 56.11

dataset	layer	title	description	start_date	end_date	data_type	bbox
tno/brogm_gld	gm_gld	Grondwaterstandonderzoek (GLD)	De meest karakteristieke (ken)gegevens van grondwaterstandonderzoeken (GLD) uit de Basisregistratie Ondergrond (BRO), gemeten in meetpunten van nederlandse grondwatermonitoringnetten (GMN). Een meetpunt vertegenwoordigt één monitoringbuis van een grondwatermonitoringput (GMW). De GLD gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 28992, 3857, 4258	4258, 1.66, 48.04, 12.43, 56.11
tno/brogm_gar	gm_gar	Grondwatersamenstellingsonderzoek (GAR)	De meest karakteristieke (ken)gegevens van grondwatersamenstellingsonderzoeken (GAR) uit de Basisregistratie Ondergrond (BRO), gemeten in meetpunten van nederlandse grondwatermonitoringnetten (GMN). Een meetpunt vertegenwoordigt één monitoringbuis van een grondwatermonitoringput (GMW). De GAR gegevenscatalogus is opgenomen op de BRO Productomgeving. Hier vindt men ook de gegevensdefinitie, met daarin onder andere een omschrijving van alle entiteiten en attributen.	NA	NA	4326, 28992, 3857, 4258	NA-4258, 1.66, 48.04, 12.43, 56.11

Zoals je ziet is er veel data beschikbaar over grondwater, zoals de kwaliteit en de grondwaterstanden. Wil je meer weten over de data is dit gedocumenteerd op https://www.bro-productomgeving.nl/bpo/latest/grondwatermonitoring. Wij gaan nu werken met de grondwaterstanden (https://www.bro-productomgeving.nl/bpo/latest/grondwatermonitoring/grondwaterstandonderzoek-gld). Deze laag heet: `gm_gld`.

Een snel overzicht van de namen van de lagen vind je met de onderstaande code

```
pdok_list_layers(pdok_search_datasets("grondwater")$id)$layer
```

```
[1] "gm_gmw"                "gm_gmw_monitoringtube" "gm_gmn"
[4] "gm_gmn_measuringpoint" "gm_gmn_reference"      "gm_gld"
[7] "gm_gar"
```

Stel dat we nu alle grondwaterstanden binnen de binnenstad van delft willen hebben, kunnen we dat eenvoudig opvragen door deze data uit PDOK te halen met de onderstaande, waarbij we filteren op basis van het gebied van de binnenstad van Delft.

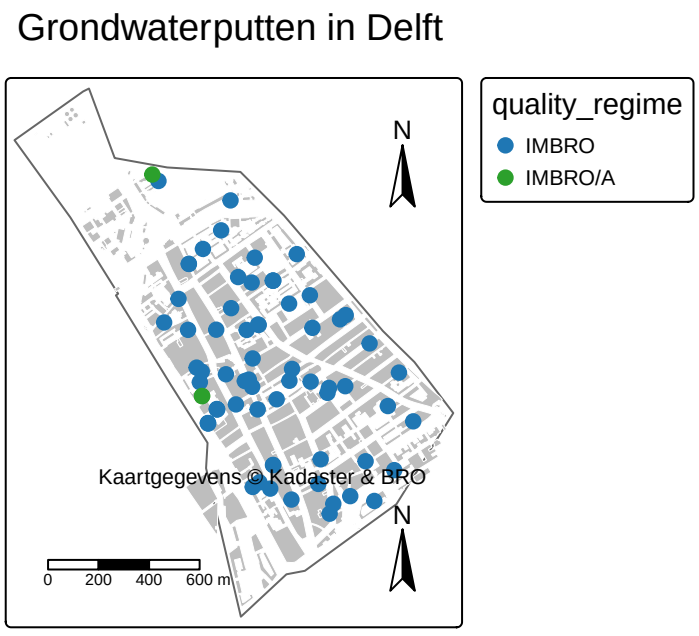
```
gw_stand_delft <- pdok_read("tno/bro-grondwatermonitoring-in-samenhang-karakteristieken",
                            "gm_gld", filter_by = binnenstad_delft)
```

Dit kunnen we nu eenvoudig op de kaart laten zien door de onderstaande code, waarbij we alle grondwaterputten in Delft laten zien en een kleur geven op basis van de

```
tmap_mode("plot")
```

```
i tmap modes "plot" - "view"
```

```
tm_basemap(pdok_basemap("grijs")) +
  tm_shape(binnenstad_delft) +
    tm_borders(col = "grey40") +
  tm_shape(gebouwen_delft) +
    tm_polygons(fill = "grey", col = "white", lwd = 0.4) +
  tm_shape(gw_stand_delft) +
    tm_dots(size = 0.5, fill = "quality_regime", fill.scale = tm_scale_categorical()) +
  tm_compass(position = c("top", "right")) +
  tm_scalebar(position = c("bottom", "left")) +
  tm_title("Grondwaterputten in Delft") +
  tm_credits("Kaartgegevens © Kadaster & BRO") +
  tm_compass()
```



4.6.1 Nu zelf aan de slag

1. Maak een kaart waarbij je de punten van de grondwaterstandmetingen kleurt op basis van de datum van het begin van met metingen.
2. Maak een kaart waarbij je de punten van de grondwaterstandmetingen kleurt op basis van de datum van het aantal metingen.
3. Maak een kaart waarbij je de punten van de grondwaterstandmetingen kleurt op basis van de duur van de metingen

4.7 Individuele meetreeksen downloaden

In de dataset `gw_stand_delft` bestaat een aantal kolommen met daarin een link naar een csv-bestand: - series_unknown_csv_url - series_preliminary_csv_url - series_fully_assessed_csv_url

Deze kolommen bevatten een aantal links. Als we de links willen zien, kunnen we bijvoorbeeld de eerste rij selecteren en dan kolommen 17,18 en 19 met `gw_stand_delft[1,17:19]`. Dat is basis-R. Je kunt ook met dplyr uit de *tidy verse* de namen van de kolommen selecteren met `gw_stand_delft[1,] |> select(series_unknown_csv_url, series_preliminary_csv_url, series_unknown_csv_url)`

```
gw_stand_delft[1,17:19]
```

```
Simple feature collection with 1 feature and 3 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 4.35 ymin: 52 xmax: 4.35 ymax: 52
Geodetic CRS:  WGS 84
# A tibble: 1 x 4
  series_unknown_csv_url           series_preliminary_c~1 series_fully_assesse~2
  <chr>                            <chr>                  <chr>
1 https://publiek.broservices.nl/~ https://publiek.brose~ https://publiek.brose~
# i abbreviated names: 1: series_preliminary_csv_url,
#   2: series_fully_assessed_csv_url
# i 1 more variable: geometry <POINT [°]>
```

```
gw_stand_delft[1,] |>
  select(series_unknown_csv_url, series_preliminary_csv_url, series_unknown_csv_url)
```

```
Simple feature collection with 1 feature and 2 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 4.35 ymin: 52 xmax: 4.35 ymax: 52
Geodetic CRS:  WGS 84
# A tibble: 1 x 3
  series_unknown_csv_url        series_preliminary_c~1    geometry
  <chr>                         <chr>                  <POINT [°]>
1 https://publiek.broservices.~ https://publiek.brose~   (4.35 52)
# i abbreviated name: 1: series_preliminary_csv_url
```

Je zou dus nu voor iedere rij de individuele dataset op kunnen vragen en kunnen downloaden, maar dat is wat veel werk. Daarom is het handig om de dataset automatisch te doorlopen. De meest eenvoudige manier is een *for loop*. Er zijn ook andere manieren, maar voor de eenvoud blijven we bij een *for loop* (wil je ook andere opties zien, ga dan naar: https://r4ds.had.co.nz/iteration.html).

De onderstaande code leest deze data in. Lees de annotatie om de code te begrijpen

```
# loop voor het doorlopen van alle links met csv-bestanden met grondwatermeting
for (i in 1:nrow(gw_stand_delft)) {
  # inlezen van de data van de link uit rij i
  gw_meting <- read.csv(gw_stand_delft$series_fully_assessed_csv_url[i])
  # maak veldnamen aan
  colnames(gw_meting) <- c("datum_tijd",  "waterstand", " StatusKwaliteitscontrole", "test4"
  # voeg een veld met het BRO-id toe
  gw_meting$bro_id <- gw_stand_delft$bro_id[i]
  # de ingelezen data is in de eerste ronde de output
  if (i==1) {
    gw_metingen_delft <- gw_meting
  } else {
    # in de volgende rondes worden de metingen samengevoegd met rbind (row-bind: combineren r
    gw_metingen_delft <- rbind(gw_metingen_delft,gw_meting)
  }
}
```

Deze code loopt helaas vast op een foutmelding, omdat sommige data niet gevonden kan worden. Daarom gaan de we `try`-functie gebruiken, om zo foutmeldingen te kunnen verwerken. Lees vooral de annotatie in de onderstaande code

```
# verdwijder eventueel bestaande metingen
if (exists("gw_metingen_delft")) {
```

```
  rm(gw_metingen_delft)
}
# loop voor het doorlopen van alle links met csv-bestanden met grondwatermeting
for (i in 1:nrow(gw_stand_delft)) {
  # probeer de data te downloaden'
  check_data <- try(read.csv(gw_stand_delft$series_fully_assessed_csv_url[i]))
  # bij een foutmelding, proberen de we volgende regel.
  if (class(check_data)=="try-error") {
    print(paste0("De dataset met BRO-id ", gw_stand_delft$bro_id[i], " mist de onderliggende
    next
  } else {
    # indien er wel een bestand gevonden wordt, kunnen we die inlezen
    gw_meting <- read.csv(gw_stand_delft$series_fully_assessed_csv_url[i])
    # maak veldnamen aan
    colnames(gw_meting) <- c("datum_tijd",  "waterstand", " StatusKwaliteitscontrole", "test
     # voeg een veld met het BRO-id toe
    gw_meting$bro_id <- gw_stand_delft$bro_id[i]
    # test of er al een samengestelde lijst met metingen bestaat, zo niet, maak deze aan
    if (!exists("gw_metingen_delft")) {
      gw_metingen_delft <- gw_meting
    } else {
      # anders samenvoegen
      gw_metingen_delft <- rbind(gw_metingen_delft,gw_meting)
    }
  }
}
```

Als het goed is, is het nu gelukt om de data in te lezen en heeft dit een aardig groot object opgeleverd met meer dan 7 miljoen (!) metingen. Belangrijk om te onthouden - Loops zijn bepaalde stappen herhaaldelijk uit te kunnen voeren - Je kunt testen of objecten bestaan in jouw workspace - Data downloaden en samenvoegen is te automatiseren en dat spaart tijd.

Het is wel fijn om nog even vast te leggen welke metingen beschikbaar zijn en welke niet. Dat kan door een nieuw veld toe te voegen met de naam `status_meting`

```
gw_stand_delft$status_meting[gw_stand_delft$bro_id %in% unique(gw_metingen_delft$bro_id)] <-
gw_stand_delft$status_meting[is.na(gw_stand_delft$status_meting)] <- "afwezig"
```

Nu we veel data gedownload hebben en dat tijd gekost heeft, is het fijn om deze data lokaal op te slaan. Dat kan met de functie `saveRDS()`. Wil je meer weten over dit formaat, kijk dan even hier https://www.geeksforgeeks.org/r-language/data-serialization-rds-using-r/.

```
# opslaan data als RDS
saveRDS(gw_metingen_delft, "data/gw_metingen_delft.rds")
```

Dit maakt het mogelijk om aangemaakte data later weer in te lezen met:

```
# inlezen rds-bestand
gw_metingen_delft <- readRDS("data/gw_metingen_delft.rds")
```

4.7.1 Nu even zelf

Check bij het werken met datum-velden of dit tekst of datum-velden zijn. Pas ze indien nodig aan (zie voorgaande colleges).

1. Maak een kaart waarop zichtbaar is welke metingen beschikbaar zijn en welke niet. Denk eraan dat een kaart altijd is voorzien van een noorpijl, schaalbalk en legenda.
2. Maak een kaart met de grondwaterstanden op 24 juni 2024 in Delft.
3. Maak een spreidingsdiagram van de grondwater op deze datum.
4. Maak een boxplot van de grondwaterstanden in juli 2021.
5. Maak een boxplot van de grondwaterstanden in januari 2021. Wat valt op?
6. Maak een kaart met de gemiddelde grondwaterstanden in mei 2023.

Je zou op basis van deze data ook volgende stappen kunnen gaat zetten ten aanzien van interpolatie. Dat gaan we nu niet behandelen.

4.8 Kades in Delft selecteren

Nu we de ruimtelijke data in R hebben ingelezen wordt het ook mogelijk om ruimtelijke vragen aan de data te stellen

Laten we eerst nog eens kijken welke lagen bestaan in de top10NL

```
pdok_list_layers("kadaster/brt-top10nl")$layer
```

```
 [1] "functioneel_gebied_multivlak"  "functioneel_gebied_punt"
 [3] "functioneel_gebied_vlak"       "gebouw_punt"
 [5] "gebouw_vlak"                   "geografisch_gebied_punt"
 [7] "geografisch_gebied_multivlak"  "geografisch_gebied_vlak"
 [9] "hoogte_lijn"                   "hoogte_punt"
[11] "inrichtingselement_lijn"       "inrichtingselement_punt"
[13] "plaats_multivlak"              "plaats_punt"
```

```
[15] "plaats_vlak"                    "plantopografie_vlak"
[17] "registratief_gebied_multivlak" "registratief_gebied_vlak"
[19] "relief_lijn"                    "relief_talud_hoge_zijde_lijn"
[21] "relief_talud_lage_zijde_lijn"  "spoorbaandeel_lijn"
[23] "spoorbaandeel_punt"             "terrein_vlak"
[25] "waterdeel_lijn"                 "waterdeel_punt"
[27] "waterdeel_vlak"                 "wegdeel_hartlijn"
[29] "wegdeel_hartpunt"               "wegdeel_lijn"
[31] "wegdeel_punt"                   "wegdeel_vlak"
```

Er bestaan verschillende lagen, waarbij punten, lijnen en vlakken (polygonen) gescheiden zijn. Het is maar net wat je nodig hebt.

We gaan nu een aantal lagen aanmaken met ruimtelijke informatie in Delft, waarbij we beginnen met de wegen en wateren om zo de kades te kunnen vinden die aan wegen grenzen. Hiertoe hebben we polygonen nodig. We hebben net gezien dat er een laag `waterdeel_vlak` en een `wegdeel_vlak` bestaat:

```
water_delft <- pdok_read("kadaster/brt-top10nl", "waterdeel_vlak", filter_by = binnenstad_del
weg_delft <- pdok_read("kadaster/brt-top10nl", "wegdeel_vlak", filter_by = binnenstad_delft)
```

```
\ Downloading PDOK features: 1135 fetched
```

Om nu te weten te komen waar wegen aan water grenst, maken we gebruik van de intersectie van deze twee lagen. We maken daarbij gebruik van de topologische relaties tussen objecten (zie ook https://r.geocompx.org/spatial-operations#topological-relations voor meer uitleg).

Het resultaat zal een lijn zijn, dus we moeten eerst onze polygonen omzetten naar lijnen, zodat de output ook een lijn zal zijn.

```
# zet de vlakgrenzen om naar een lijn
watergrenzen_delft <- st_cast(st_boundary(water_delft), "MULTILINESTRING")
weggrenzen_delft <- st_cast(st_boundary(weg_delft), "MULTILINESTRING")
# bepaal de intersectie van weg en water en sla dit op als een object
wegkade_delft <- st_intersection(watergrenzen_delft, weggrenzen_delft)
```

```
Warning: attribute variables are assumed to be spatially constant throughout
all geometries
```

Deze kades kunnen we nu op de kaart zetten, zie onderstaand:

```
tmap_mode("plot")
```

```
i tmap modes "plot" - "view"
```

```
tm_basemap(pdok_basemap("grijs")) +
  tm_shape(binnenstad_delft) +
    tm_borders(col = "grey40") +
  tm_shape(gebouwen_delft) +
    tm_polygons(fill = "#d95f02", col = "white", lwd = 0.4) +
  tm_shape(wegkade_delft) +
    tm_lines(col = "blue") +
  tm_title("Gebouwen en kades in Delft") +
  tm_credits("Kaartgegevens © Kadaster") +
  tm_compass(position = c("top", "right")) +
  tm_scalebar(position = c("bottom", "left"))
```

```
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```



4.8.1 Nu zelf aan de slag

1. Bepaal de totale lengte van de kades langs wegen in Delft. Je kunt hiervoor gebruik maken van `st_length()`. Dit geeft alvast een indicatie ten aanzien van mogelijk onderhoud in deze stad.
2. Als het goed is, heb je gezien dat nog niet alle kades zijn gevonden. Dat komt omdat er ook gebouwen grenzen aan water. Bepaal ook deze kades.
3. Welke kades missen nog? Zoek de juiste lagen op in PDOK om dit te bepalen.
4. Het openbaar vervoer is ook in PDOK te vinden. Welke busroutes lopen door de stad? Kun je die ook op de kaart zetten?
5. De hoogte van de kades is nu niet bekend, maar het zou mogelijk moeten zijn om deze te kunnen achterhalen op basis van open data. Hoe zou je dat aanpakken? Kun je dat ook in R?

4.9 Tot slot

Je hebt in deze les geleerd om

- data uit PDOK te zoeken
- lagen uit PDOK in R in te laden
- met loops te werken
- ruimtelijke bewerkingen uit te voeren
- een start te maken met ruimtelijke analyses.

References

Eisma, Coen. 2026. *Pdokr: Access Open Geodata from the Dutch 'PDOK' Platform*. https://github.com/coeneisma/pdokr.

Grolemund, Garrett, and Hadley Wickham. 2011. "Dates and Times Made Easy with Lubridate." *Journal of Statistical Software* 40 (3): 125. https://www.jstatsoft.org/v40/i03/.

Kassambara, Alboukadel. 2023. *Ggpubr: 'Ggplot2' Based Publication Ready Plots*. https://rpkgs.datanovia.com/ggpubr/.

Pebesma, Edzer. 2018. "Simple Features for r: Standardized Support for Spatial Vector Data." *The R Journal* 10 (1): 439–46. https://doi.org/10.32614/RJ-2018-009.

Pebesma, Edzer, Martin Fleischmann, Josiah Parry, et al. n.d. *Spatial Data Science Languages: Commonalities and Needs*. https://doi.org/10.48550/arXiv.2503.16686.

Tennekes, Martijn. 2018. "Tmap: Thematic Maps in r." *Journal of Statistical Software* 84 (6): 139. https://doi.org/10.18637/jss.v084.i06.

Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. http://ggplot2.org.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan. 2023. *Dplyr: A Grammar of Data Manipulation*. https://dplyr.tidyverse.org.

Wickham, Hadley, Jim Hester, and Jennifer Bryan. 2026. *Readr: Read Rectangular Text Data*. https://readr.tidyverse.org.

Wickham, Hadley, Davis Vaughan, and Maximilian Girlich. 2023. *Tidyr: Tidy Messy Data*. https://tidyr.tidyverse.org.